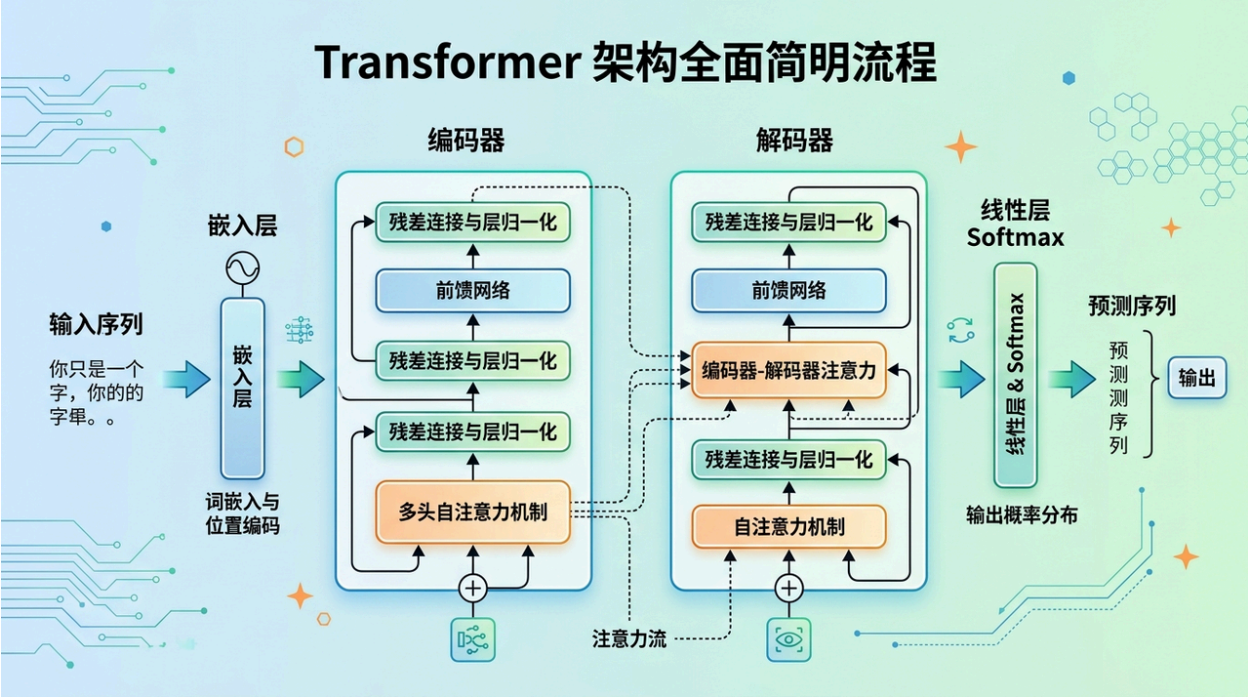


大模型Transformer架构从0-1架构深度解析

🔥 【Transformer入门到实战】 transformer基本介绍和常见激活函数

一、Transformer到底是个啥？

简单来说，Transformer就是一种神经网络架构，就像盖房子的图纸一样。2017年Google的研究人员在论文《Attention is All You Need》中提出了它，从此改变了整个AI界。



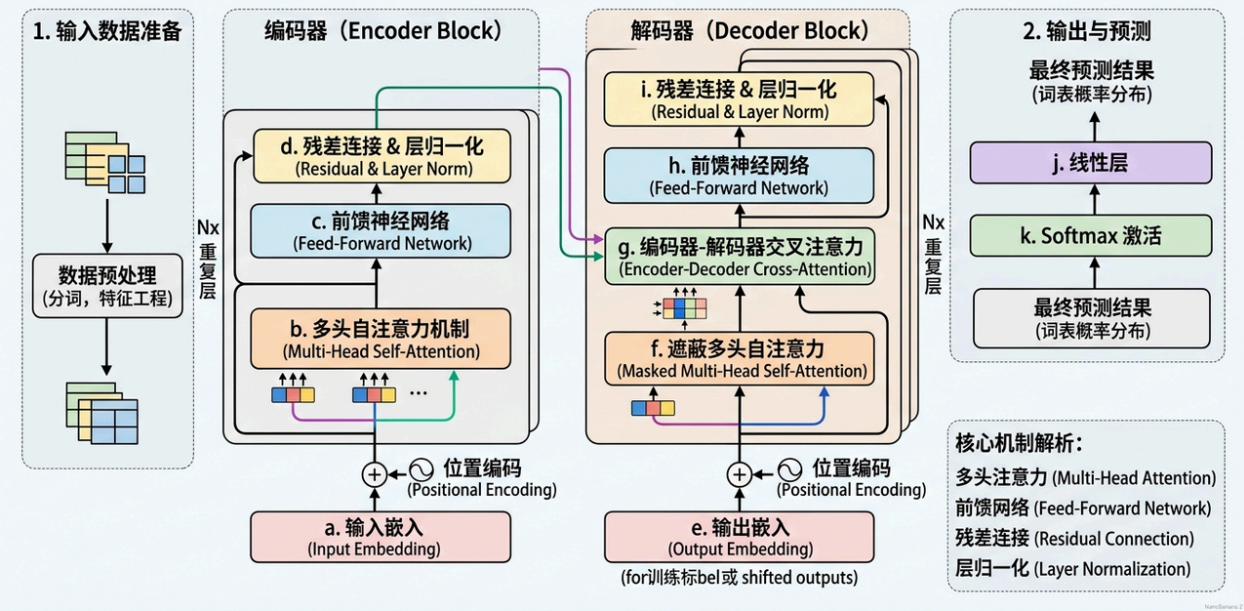
1.1 Transformer解决了什么问题？

🔥 在Transformer出现之前，处理文本主要用RNN(循环神经网络)和LSTM。但这些模型有个大问题：处理长文本时太慢了！

举个例子：

- 你要翻译一句话：“我今天早上吃了一个苹果”
- 传统RNN要这样处理：先看“我”，再看“今天”，再看“早上”...一个字一个字按顺序来
- 这就像你排队买奶茶，前面的人不走，你就得一直等
- 架构

Transformer 模型多架构设计全景图（清晰流程）



而Transformer用了**自注意力机制(Self-Attention)**，可以同时看所有的字，就像开了很多个窗口，大家一起办业务，效率高多了！

1.2 Transformer的核心思想

Transformer的核心是**"注意力机制"**。什么意思呢？

想象你在读一篇文章：

- 当你看到"它"这个字时，你的大脑会自动往前找，"它"指的是什么
- 可能是前面提到的"猫"，也可能是"汽车"
- 你的大脑会自动"注意"到相关的词

Transformer就是模仿这个过程，让模型学会关注句子中最重要的部分。

👑 传统RNN vs Transformer注意力机制

传统RNN - 顺序处理 🐢

我 爱 吃 苹果



Transformer - 并行注意力 🚀

我 爱 吃 苹果



🕒 像排队一样，必须一个一个处理

😓 句子越长，等待时间越久

❌ 容易忘记前面的信息

@yopai

⚡ 同时看所有词，并行处理

🎯 自动找到词与词之间的关系

✅ 速度快，效果好

二、Transformer和大模型是什么关系？

2.1 简单类比

- Transformer = 建筑设计图纸
- 大模型 = 用这个图纸建出来的摩天大楼

更具体地说：

- Transformer是架构，告诉你神经网络应该怎么搭建
- 大模型是用这个架构训练出来的具体模型

2.2 著名的大模型都用Transformer

看看这些你肯定听过的名字：

1. GPT系列 (GPT-3, GPT-4, ChatGPT)

- 只用了Transformer的**解码器(Decoder)**部分
- 擅长生成文本、对话、写代码

2. BERT

- 只用了Transformer的**编码器(Encoder)**部分
- 擅长理解文本、分类、问答

3. T5、BART

- 用了完整的Transformer(Encoder + Decoder)
- 擅长翻译、摘要等任务

2.3 训练大模型的过程

大模型使用了Transformer架构训练过程是这样的：

- 1. 准备数据：收集海量文本(比如整个互联网的文章)
- 2. 搭建架构：按照Transformer设计搭建神经网络
- 3. 开始训练：让模型不断学习，调整参数
- 4. 得到大模型：训练好后就能用了

就像：

- Transformer = 健身房的器材和训练计划
- 训练过程 = 你每天去健身
- 大模型 = 练出来的好身材



三、神经网络是什么？

在讲激活函数之前，我们得先理解什么是神经网络。

3.1 人脑神经元的启发

人的大脑有大约860亿个神经元，它们互相连接，传递信息。神经网络就是模仿这个原理！

一个神经元的工作原理：

1. **接收信号**：从其他神经元接收电信号
2. **处理信号**：把这些信号加起来
3. **决定是否激活**：如果信号够强，就"点亮"，传给下一个神经元

3.2 人工神经元

计算机里的神经元是这样工作的：

输入1 × 权重1 + 输入2 × 权重2 + 输入3 × 权重3 + 偏置 = 输出

举个实际例子，判断要不要出门买奶茶：

- **输入1**：天气好不好 (0-10分)
- **输入2**：有多渴 (0-10分)
- **输入3**：钱包里有多少钱 (0-10分)

每个输入都有一个**权重**（重要性）：

- 天气权重 = 0.3 (不太重要)
- 渴的程度权重 = 0.5 (比较重要)
- 钱的数量权重 = 0.2 (不太重要)

计算：

决策分数 = 天气 × 0.3 + 渴 × 0.5 + 钱 × 0.2 + 偏置

如果分数 > 5，就去买奶茶！

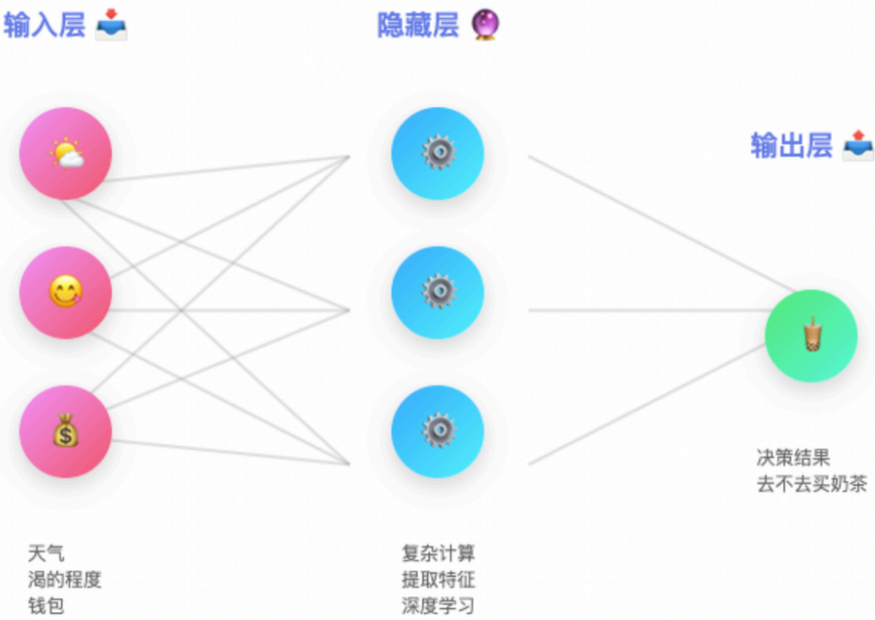
3.3 神经网络 = 很多神经元连在一起

一个神经元只能做简单判断，但把成千上万个神经元连起来，分成好几层，就能处理超级复杂的任务！

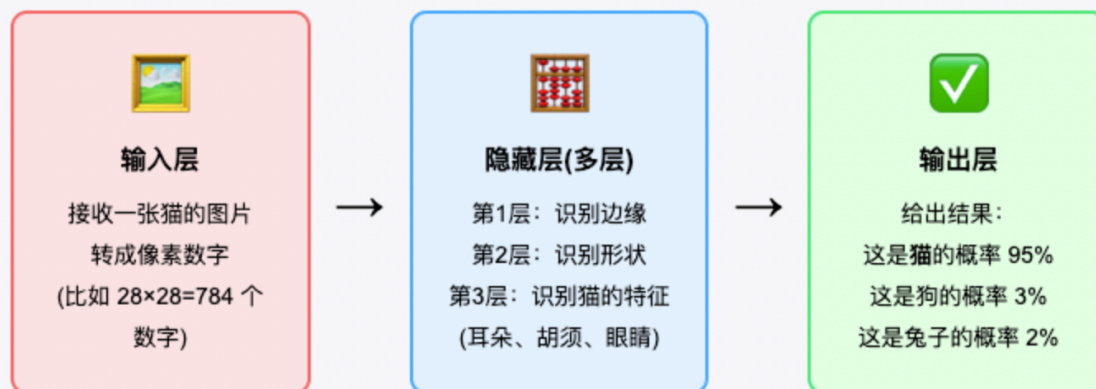
典型的三层结构：

1. **输入层**：接收原始数据
2. **隐藏层**：进行复杂计算（可以有很多层）
3. **输出层**：给出最终结果

神经网络结构详解



💡 实际应用举例：图片识别



@yopai

🎯 关键点

- ✓ **神经网络** = 模仿大脑的计算模型
- ✓ **神经元** = 最基本的计算单元, 接收输入→加权求和→激活→输出
- ✓ **层数越多** = 能处理越复杂的任务(这就是"深度学习"的"深度")
- ✓ **权重和偏置** = 训练过程就是不断调整这些参数, 让模型更准确

四、激活函数：神经网络的灵魂

激活函数是神经网络中非常重要的部分。

4.1 为什么需要激活函数？

不用激活函数会怎样？

如果没有激活函数，神经网络就只能做线性计算：

$$y = w_1 \times x_1 + w_2 \times x_2 + w_3 \times x_3 + b$$

这样不管你堆多少层，本质上都等于一个简单的线性函数！就像：

- 1层线性 = $y = 2x + 1$
- 100层线性堆叠 = 还是 $y = \text{某个数} \times x + \text{某个数}$

这太简单了，根本处理不了复杂问题！

有了激活函数之后：

激活函数引入了**非线性**，让神经网络可以学习复杂的模式。就像：

- 线性 = 只能画直线
- 非线性 = 可以画曲线、圆、各种复杂图形

4.2 常见的激活函数

1. Sigmoid 📊

$$\sigma(x) = 1 / (1 + e^{-x})$$



特点：输出在0到1之间，像S型曲线

✅ 优点

- 输出可解释为概率
- 平滑可导
- 适合二分类

❌ 缺点

- 梯度消失严重
- 计算复杂(指数)
- 输出不以0为中心

🔧 常用场景

二分类输出层、需要概率输出的场景

@yopai

2. ReLU ⭐ (最常用)

$$f(x) = \max(0, x)$$



特点：负数变0，正数不变，超简单！

✅ 优点

- 计算超级快
- 缓解梯度消失
- 训练效果好
- 生物学合理性

❌ 缺点

- Dead ReLU问题
- 输出不以0为中心

🔧 常用场景

几乎所有隐藏层的首选！CNN、深度网络默认选择

4. Sigmoid函数

公式： $\sigma(x) = 1 / (1 + e^{(-x)})$

特点：

- 输出范围：0到1之间
- 形状：S形曲线
- 可以理解为"概率"

形象理解： 就像一个温柔的开关：

- 当输入很小时(负数)，输出接近0 = "关"
- 当输入很大时(正数)，输出接近1 = "开"
- 中间过渡是平滑的

什么时候用？

- 二分类问题的输出层(判断是或否)
- 需要输出概率的时候

缺点：

- 容易梯度消失(训练变慢)
- 计算相对慢

5. ReLU (Rectified Linear Unit) - 最常用！

公式： $f(x) = \max(0, x)$

特点：

- 输入为负数时，输出0
- 输入为正数时，输出就是输入本身

形象理解： 就像一个严格的门卫：

- 负面情绪(负数)一律拦住 = 输出0
- 正面能量(正数)直接放行 = 输出原值

为什么这么受欢迎？

- 计算超快(只需要比较大小)
- 缓解梯度消失问题
- 训练效果好

什么时候用？

- 隐藏层的默认选择
- 几乎所有的深度学习模型

缺点：

- "Dead ReLU"问题：有些神经元可能永远输出0

3. Leaky ReLU 🔧

$$f(x) = \max(0.01x, x)$$



特点：负数时有小斜率，不会完全死亡

✅ 优点

- 解决Dead ReLU
- 保留ReLU优点
- 允许负梯度

❌ 缺点

- 需要调参(斜率)
- 效果不总是更好

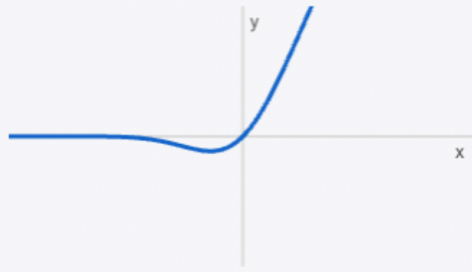
🔧 常用场景

当ReLU出现神经元死亡时的替代方案

@yopai

4. GELU ★ (Transformer专用)

$$f(x) = x \cdot \Phi(x)$$



特点：平滑非线性，结合概率思想

✅ 优点

- 平滑可导
- 性能优秀
- Transformer标配
- 更符合自然分布

❌ 缺点

- 计算稍复杂
- 需要更多算力

🔧 常用场景

GPT、BERT等Transformer模型的首选！NLP任务

6. Leaky ReLU - ReLU的改进版

公式： $f(x) = \max(0.01x, x)$

特点：

- 负数时不是完全为0，而是一个很小的负数(0.01x)

形象理解：比ReLU温柔一点的门卫：

- 负面情绪不是完全拦住，而是让它稍微进来一点点

优点：

- 解决了Dead ReLU问题
- 保留了ReLU的优点

7. GELU (Gaussian Error Linear Unit) - Transformer最爱！

公式： $f(x) = x \times \Phi(x)$ (其中 $\Phi(x)$ 是高斯分布的累积分布函数)

特点：

- 更平滑的曲线
- 结合了概率的思想

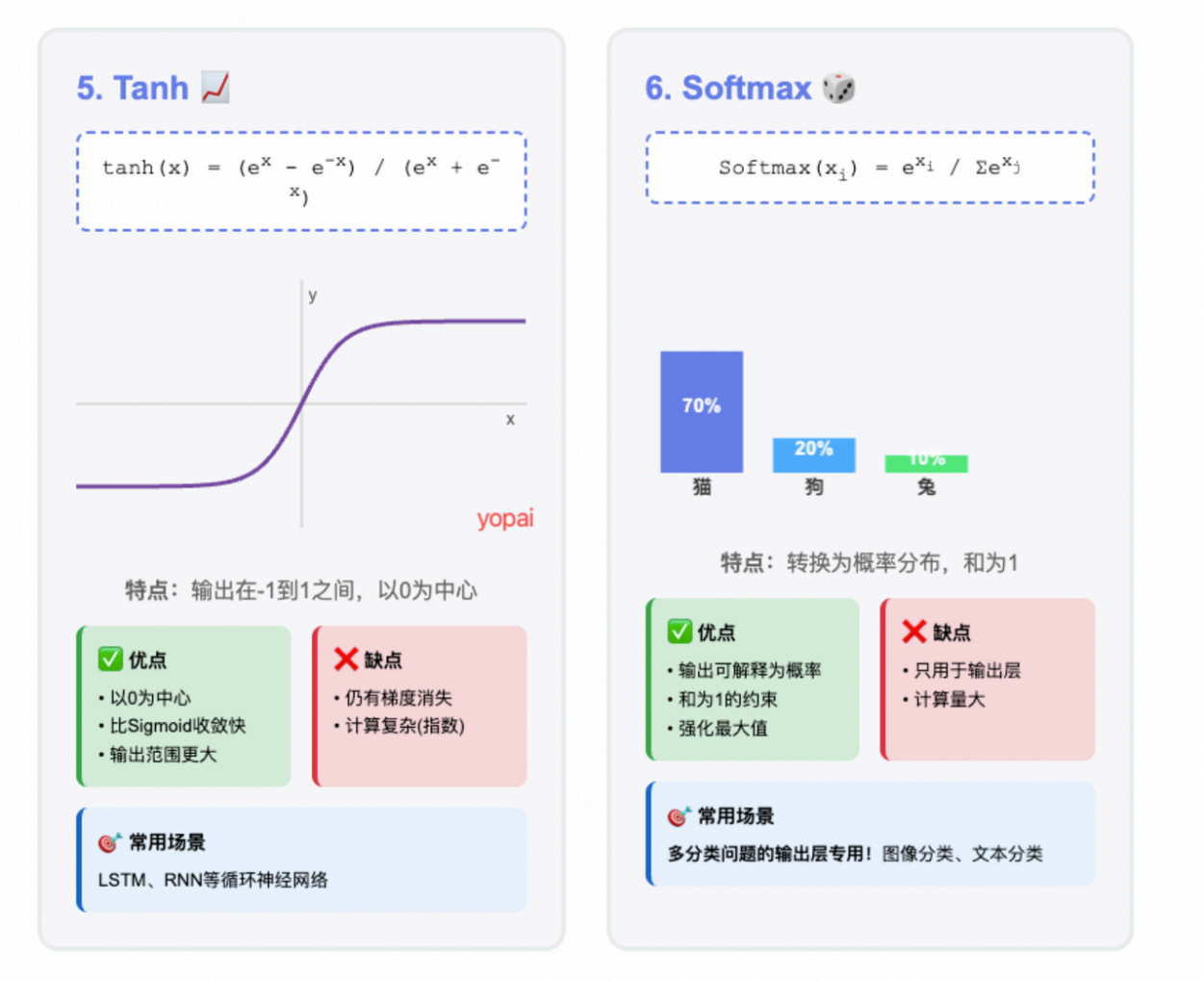
形象理解：就像一个会思考的智能门卫：

- 不只看正负，还看"有多正"或"有多负"

- 决策更细腻、更智能

为什么Transformer用它？

- 训练效果更好
- 更符合自然语言的分布特征
- GPT、BERT都在用！



9. Softmax - 多分类专用

公式： $\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum e^{x_j}}$

特点：

- 把一堆数字转成概率分布
- 所有输出加起来=1

形象理解： 就像评委打分：

- 输入： [猫:5分, 狗:2分, 兔子:1分]
- 输出： [猫:70%, 狗:24%, 兔子:6%]
- 把分数变成百分比，加起来刚好100%

什么时候用？

- 多分类问题的输出层
- 需要概率分布的时候



任务类型	推荐激活函数	使用位置	热度
深度CNN	ReLU / Leaky ReLU	隐藏层	★★★★★
Transformer/NLP	GELU	隐藏层	★★★★★
二分类	Sigmoid	输出层	★★★★☆
多分类	Softmax	输出层	★★★★★
RNN/LSTM	Tanh	隐藏层	★★★★☆

五、总结：把所有知识串起来

 知识框架

1. Transformer是什么？

- 一种神经网络架构（设计图纸）
- 基于自注意力机制
- 可以并行处理，速度快

2. Transformer和大模型的关系

- Transformer = 架构设计
- 大模型 = 用这个架构训练出来的成品
- GPT、BERT都是基于Transformer

3. 神经网络基础

- 模仿人脑神经元
- 由输入层、隐藏层、输出层组成

- 通过权重和偏置进行计算

4. 激活函数的作用

- 引入非线性
- 让网络能学习复杂模式
- 不同场景选择不同的激活函数

如果你要做NLP任务（比如训练一个小型语言模型）：

架构：Transformer

隐藏层激活函数：GELU

输出层：Softmax（分类）或Linear（生成）

如果你要做图像识别：

架构：CNN

隐藏层激活函数：ReLU

输出层：Softmax

如果你遇到训练问题：

- ReLU导致神经元死亡 → 试试Leaky ReLU
- 训练太慢 → 检查是不是用了Sigmoid/Tanh在隐藏层
- Transformer效果不好 → 确认是否用了GELU

深度理解Transformer的注意力机制

一、先搞清楚：PyTorch和Transformer到底啥关系？

很多刚入门的朋友容易搞混这两个概念，但是压根不是一个层面的东西：

PyTorch是什么？

- 它是一个深度学习**框架**，就像你盖房子用的工具箱
- 提供了张量计算、自动求导、GPU加速等基础功能
- 你可以用它来实现各种神经网络模型

Transformer是什么？

- 它是一种神经网络**架构**，是一种具体的模型设计方案
- 就像房子的设计图纸，规定了怎么组织各个组件
- 可以用PyTorch、TensorFlow等任何框架来实现

打个比方：

- PyTorch = 锤子、钉子、电钻（工具）
- Transformer = 房子设计图（架构方案）
- 你用PyTorch这套工具，按照Transformer的设计图，把模型搭建出来

所以当你听到"用PyTorch实现Transformer"，意思就是用PyTorch这个工具箱，按照Transformer的架构把模型代码写出来。

二、AI建模的核心：特征抽取器

什么是特征抽取？

咱们先从一个简单例子说起。假设你要教电脑识别猫和狗：

原始数据： 一张图片（比如256×256像素，3个颜色通道） **最终目标：** 判断是猫还是狗

中间这个把"原始像素"转化成"可以用来分类的有用信息"的过程，就叫**特征抽取**。

传统方法的困境：

原始像素 → 手工设计的特征（边缘、纹理、颜色等） → 分类器

↑ 这一步需要专家经验，很难

深度学习的突破：

原始数据 → 神经网络（自动学习特征） → 输出结果

↑ 这就是特征抽取器！

为什么说Transformer是最核心的特征抽取器？

在深度学习发展史上，特征抽取器经历了几个时代：

- 1. **CNN时代（2012-2017）：** 卷积神经网络，擅长处理图像
- 2. **RNN/LSTM时代（2014-2017）：** 循环神经网络，擅长处理序列
- 3. **Transformer时代（2017-至今）：** 统治级架构，横扫NLP、CV、语音等所有领域

Transformer为什么这么牛？

因为它有一个超级武器——**注意力机制（Attention）**！这个机制让模型能够：

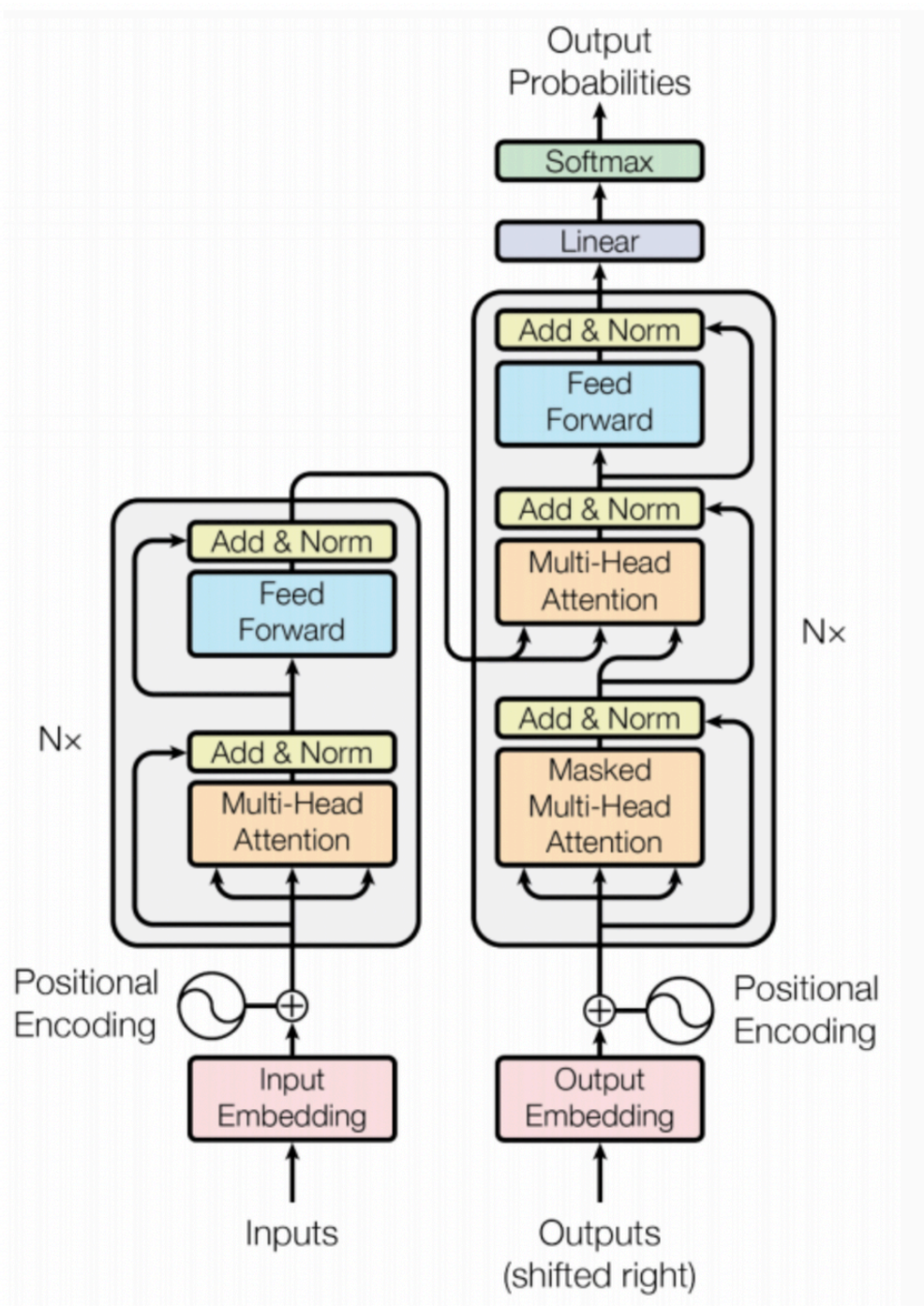
- 1. **并行处理：** 不像RNN那样必须一个一个字处理，Transformer可以同时看所有输入
- 2. **长距离依赖：** 轻松捕捉句子中相隔很远的词之间的关系
- 3. **动态权重：** 根据上下文动态调整每个词的重要性

举个例子：

句子："银行账户里的钱不够了"

- 传统方法：按顺序处理，"银行"的信息可能传到"钱"时已经衰减了
- Transformer：同时看整个句子，立刻知道"银行"指的是金融机构而不是河岸

这就是为什么GPT、BERT、DALL-E、Stable Diffusion等顶级模型全都基于Transformer架构！



三、Transformer整体架构详解

好，现在进入重头戏！咱们来看看Transformer长什么样。

3.1 高层视角：黑盒模型

最简单的理解：Transformer就是一个"翻译机器"

- **输入：** 一句法语
- **输出：** 对应的英语翻译

| Encoder-Decoder Attention: 关注编码...



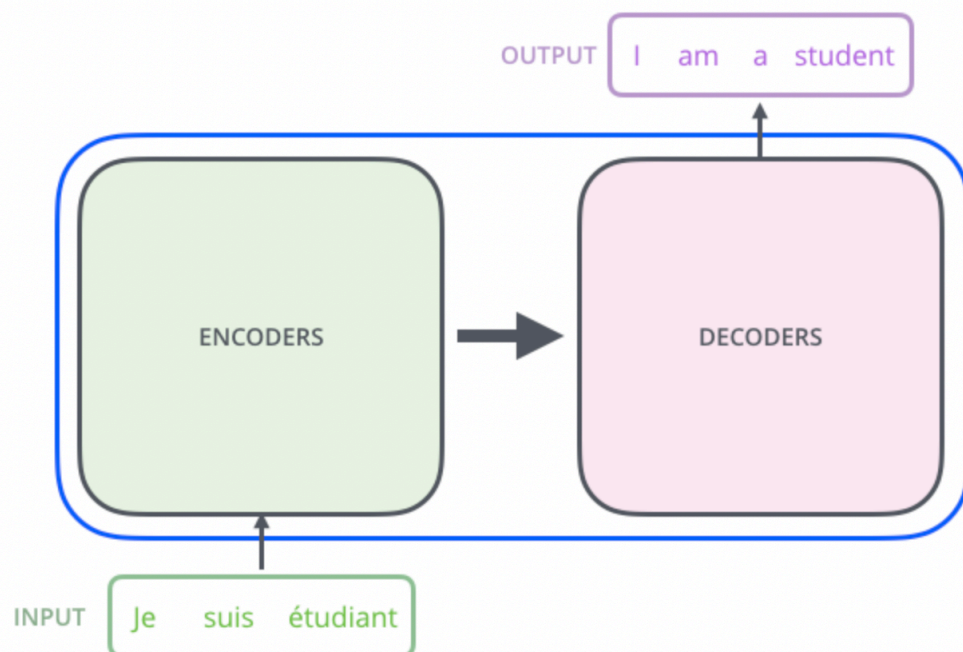
用户914659 4月8日 09:43
Multi-Head Attention



用户914659 4月8日 09:44
Query (Q) : 来自 Decoder 上一层
Key (K) : 来自 Encoder 输出



3.2 打开黑盒：编码器-解码器结构



Transformer由两大部分组成：

1. 编码器（Encoder）：理解输入的含义
2. 解码器（Decoder）：生成输出

这就像你听到法语（编码器理解），然后用英语说出来（解码器生成）。

3.3 堆叠结构：6层编码器 + 6层解码器

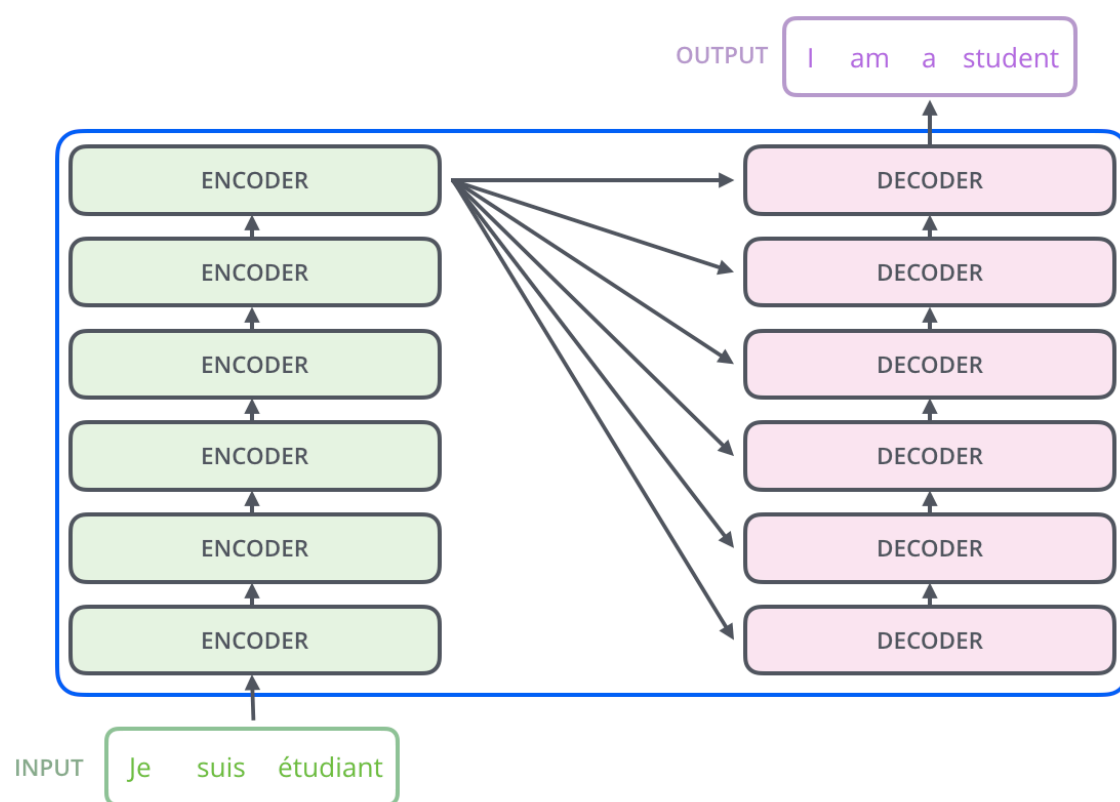
Value (V)：来自 Encoder 输出



用户914659 4月8日 09:48

Encoder-Decoder Attention = Decoder 在“读题”（看 Encoder 输出）

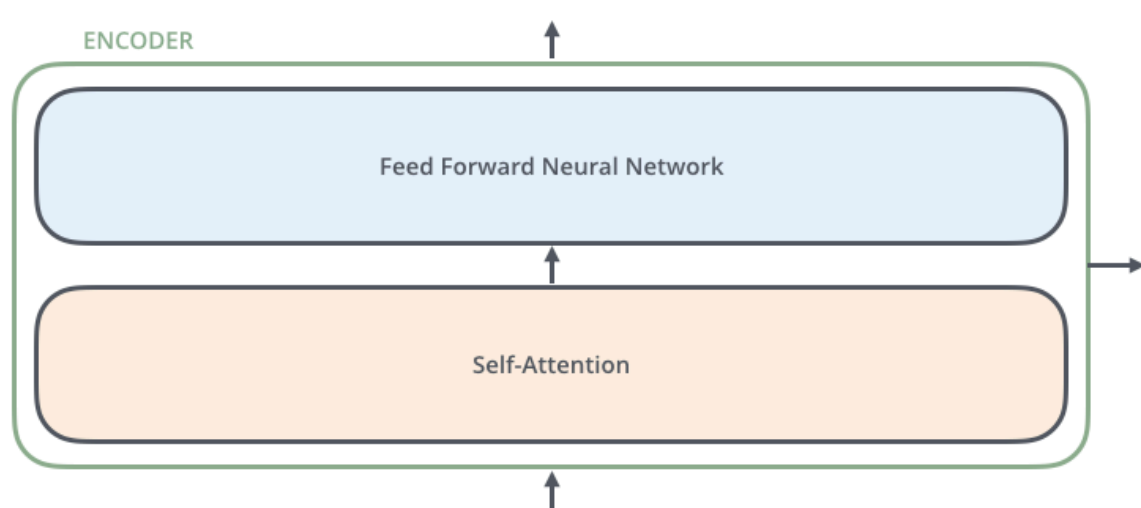
- Encoder：把原文编码成语义表示
- Decoder：每生成一个词，都去“查”原文哪里相关
为什么 Encoder 没有这一层？
- 因为：
- Encoder 只处理输入（不需要参考别的序列）
- Decoder 需要：
- 看自己（Self-Attention）
- 看输入（Encoder-Decoder Attention）



原始论文中：

- 6个相同结构的编码器堆叠起来（不共享权重）
- 6个相同结构的解码器堆叠起来
- 为什么是6？没有特别原因，你可以试试其他数字

3.4 编码器内部结构



每个编码器有两个子层：

1. Self-Attention层（自注意力层）：

- 让模型在处理每个词时，能看到句子中的其他所有词

- 这是核心！后面会详细讲

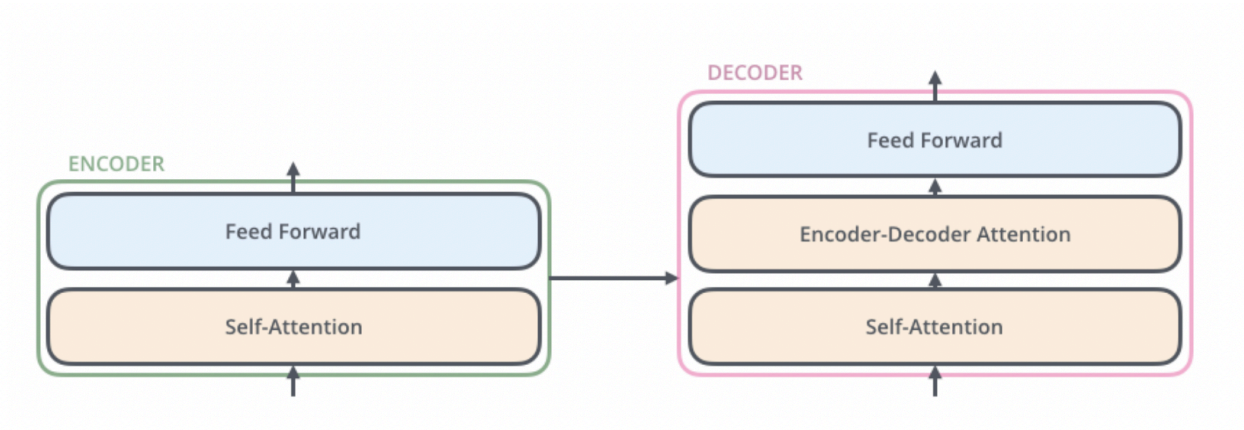
2. Feed Forward层（前馈神经网络）：

- 就是普通的全连接神经网络
- 对每个位置独立应用（位置之间不共享信息）

数据流动过程：

输入词 → Embedding（词向量）
↓
Self-Attention（关注上下文）
↓
Add & Norm（残差连接+归一化）
↓
Feed Forward（非线性变换）
↓
Add & Norm
↓
输出到下一层编码器

3.5 解码器内部结构



解码器比编码器多了一层，有三个子层：

1. **Masked Self-Attention：** 只能看到当前位置之前的词（不能偷看后面的答案）
2. **Encoder-Decoder Attention：** 关注编码器的输出（读取原文信息）
3. **Feed Forward：** 和编码器一样

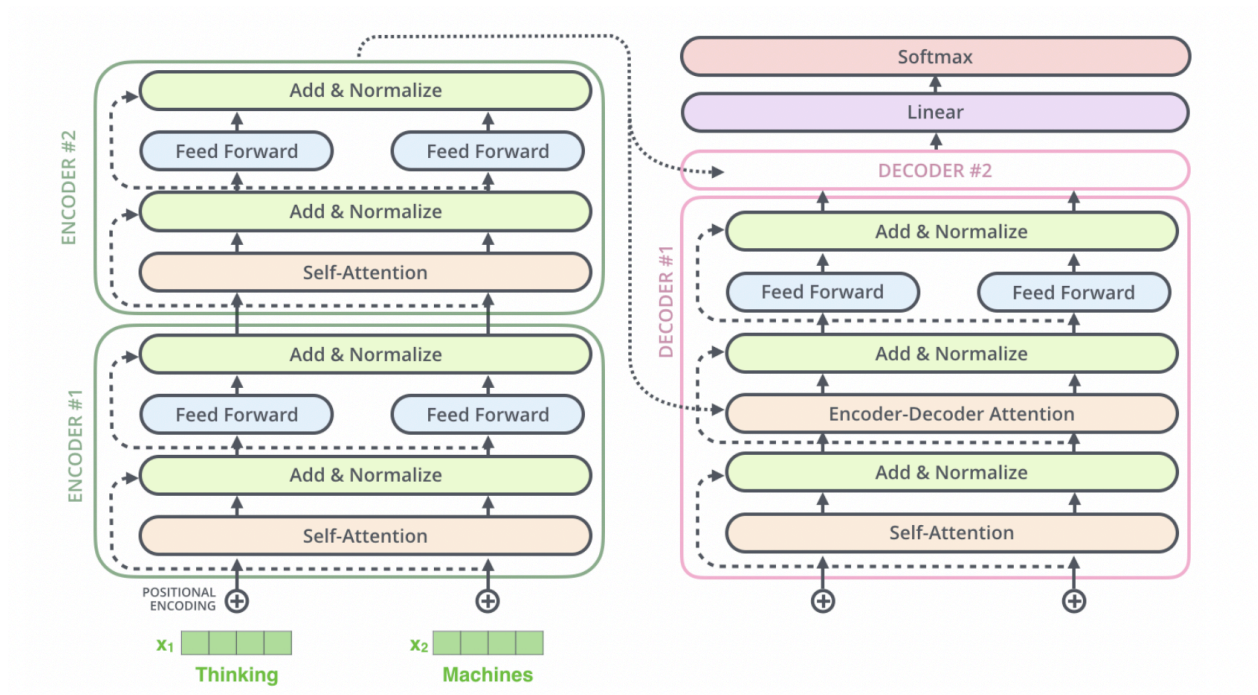
为什么要Mask？

假设你在翻译"我爱你"→"I love you"：

- 生成"I"时：只能基于"我"
- 生成"love"时：可以看"我爱"和已生成的"I"
- 生成"you"时：可以看"我爱你"和"I love"

不能让模型在生成"I"的时候就看到答案里的"love"，那不就作弊了吗？

3.6 完整数据流



训练时的完整流程：

1. 输入句子 → Embedding + 位置编码 → 进入编码器
2. 编码器逐层处理，提取特征
3. 编码器输出 → 传给解码器的Encoder-Decoder Attention
4. 解码器逐词生成翻译
5. 最后Linear层 + Softmax → 输出概率分布

四、Decoder-Only架构的应用（GPT系列的秘密）

现在最火的**GPT系列**（ChatGPT、GPT-4等）用的不是完整的Transformer，而是**只用了解码器（Decoder-Only）**！

4.1 为什么只用Decoder？

原始Transformer是为了**翻译任务**设计的：

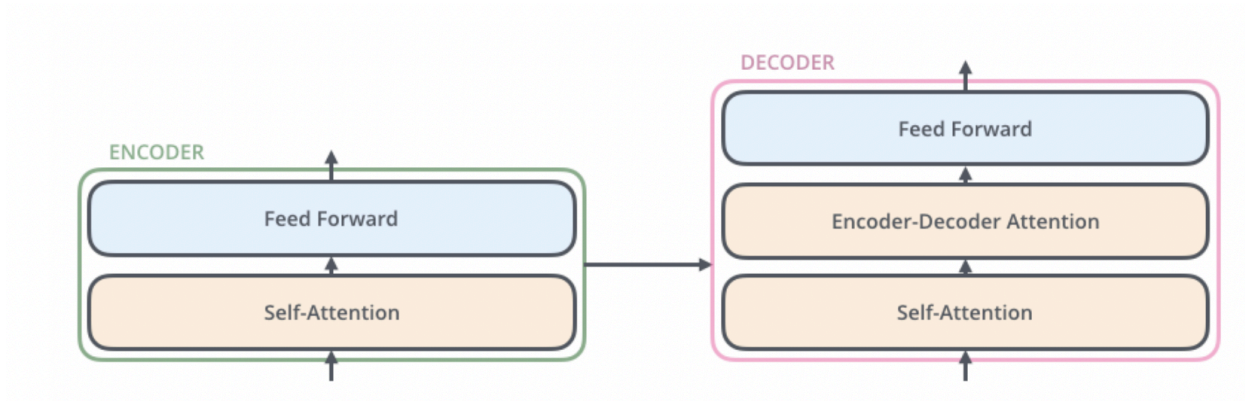
- 编码器：理解源语言
- 解码器：生成目标语言

但对于**文本生成任务**（比如写文章、对话），我们只需要：

- 输入：一段文本提示
- 输出：续写的内容

这种情况下，不需要分开的编码器和解码器，只用解码器就够了！

4.2 Decoder-Only的工作原理



核心思想： 把输入和输出看成同一个序列

举个例子：

输入提示："今天天气"

模型处理："今天 天气 真 好 啊"

↑ 输入 ↑ ↔ ↑ 输出 ↑

模型在生成"真"的时候：

- 可以看到"今天 天气"（输入）
- 不能看到"好 啊"（未来的输出）

关键配置：

- 使用Masked Self-Attention（遮住未来信息）
- 没有Encoder-Decoder Attention层
- 堆叠N个相同的Decoder Block

著名的Decoder-Only模型：

- **GPT系列：** GPT-2、GPT-3、GPT-4、ChatGPT
- **LLaMA系列：** Meta开源的大模型
- **Claude：**
- **PaLM：** Google的大模型

4.3 Decoder-Only vs 完整Transformer

特性	完整Transformer	Decoder-Only
典型应用	机器翻译	文本生成、对话
结构	Encoder + Decoder	仅Decoder
输入输出	输入和输出是不同序列	输入输出是同一序列
训练目标	Seq2Seq（序列到序列）	语言建模（预测下一个词）
代表模型	BERT（只用Encoder）、T5	GPT系列、LLaMA

五、注意力机制图解（Self-Attention的魔法）

好了，终于到最核心的部分了！注意力机制到底是什么？

5.1 直观理解：什么是"注意力"？

| score_1 = Q_thinking · K_thinking # 自...



电商用户1221 7月16日 18:42

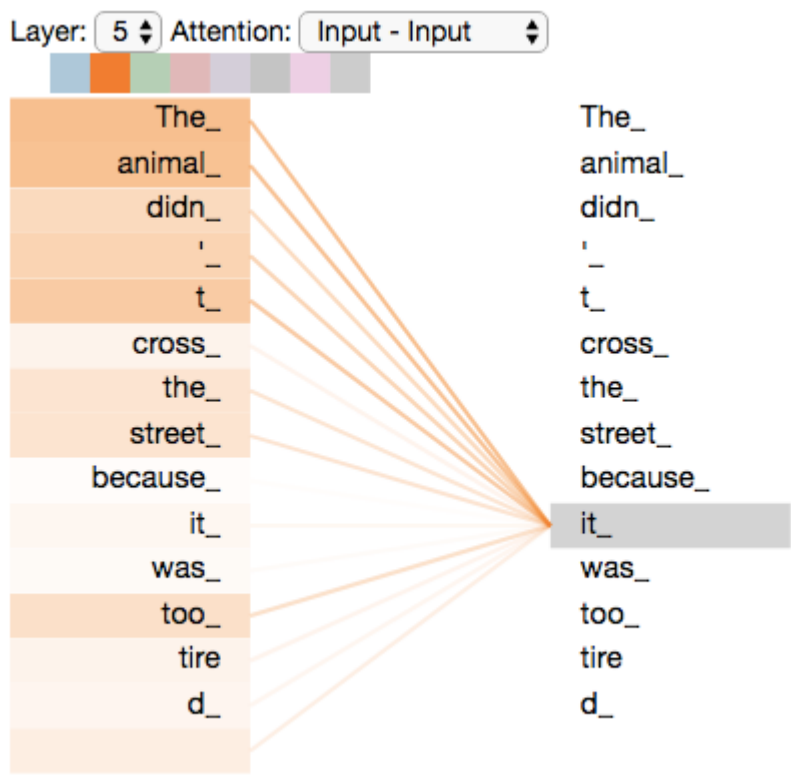
这里是乘以 K 的转置

咱们先看一个经典例子：

句子： "The animal didn't cross the street because it was too tired" **问题：** "it"指的是什么？

人类秒懂：it = animal （而不是street）

但对于计算机，怎么让它理解这种关系呢？这就需要注意力机制！



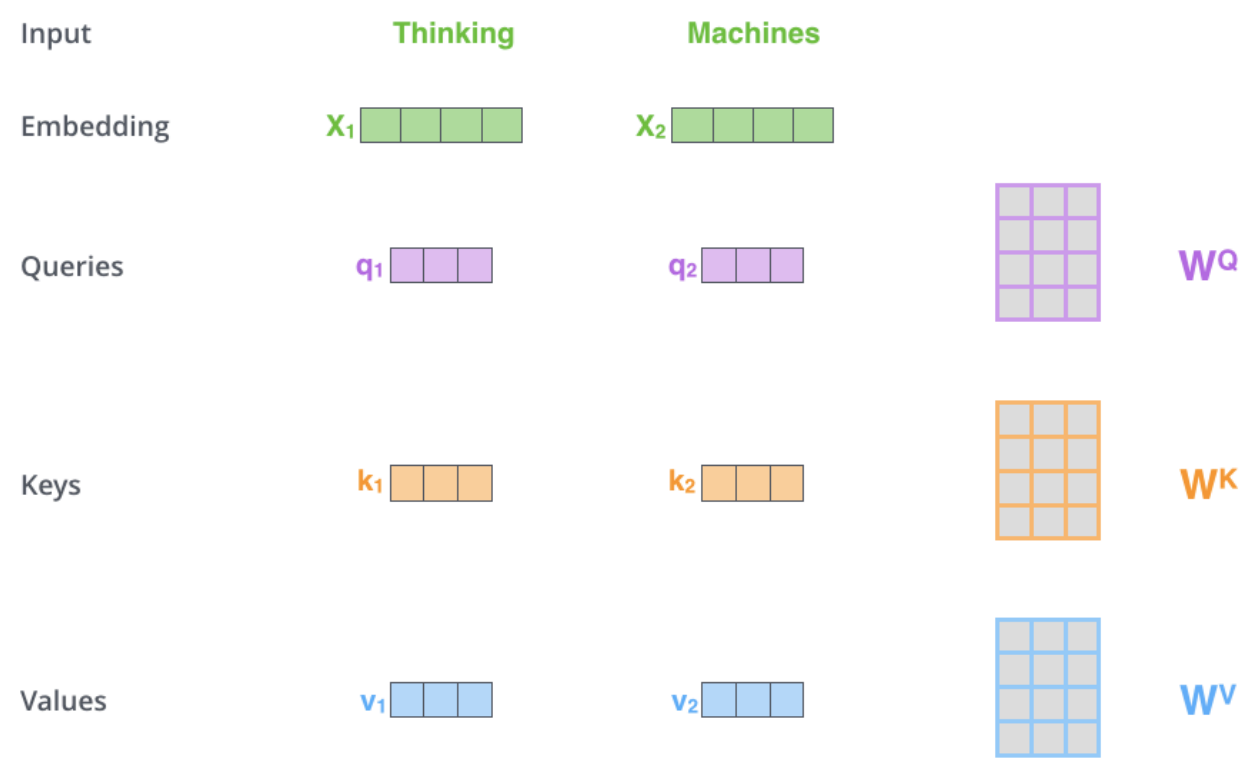
当模型处理"it"这个词时：

- 注意力机制让它"关注"句子中的其他词
- 发现"animal"和"it"的关系最强
- 把"animal"的信息融入到"it"的表示中

5.2 注意力计算的直观过程

假设我们在处理句子："Thinking Machines"

Step 1：为每个词创建三个向量



对于每个词（比如"Thinking"），我们创建三个向量：

- **Query（查询）**： "我想查询什么信息？ "
- **Key（键）**： "我能提供什么信息？ "
- **Value（值）**： "我的具体信息内容是什么？ "

类比：图书馆检索系统

- Query = 你输入的搜索关键词： "人工智能"
- Key = 每本书的索引标签： "机器学习"、"深度学习"、"AI"...
- Value = 书的实际内容

怎么得到这三个向量？

X = 词的embedding向量（维度：512）

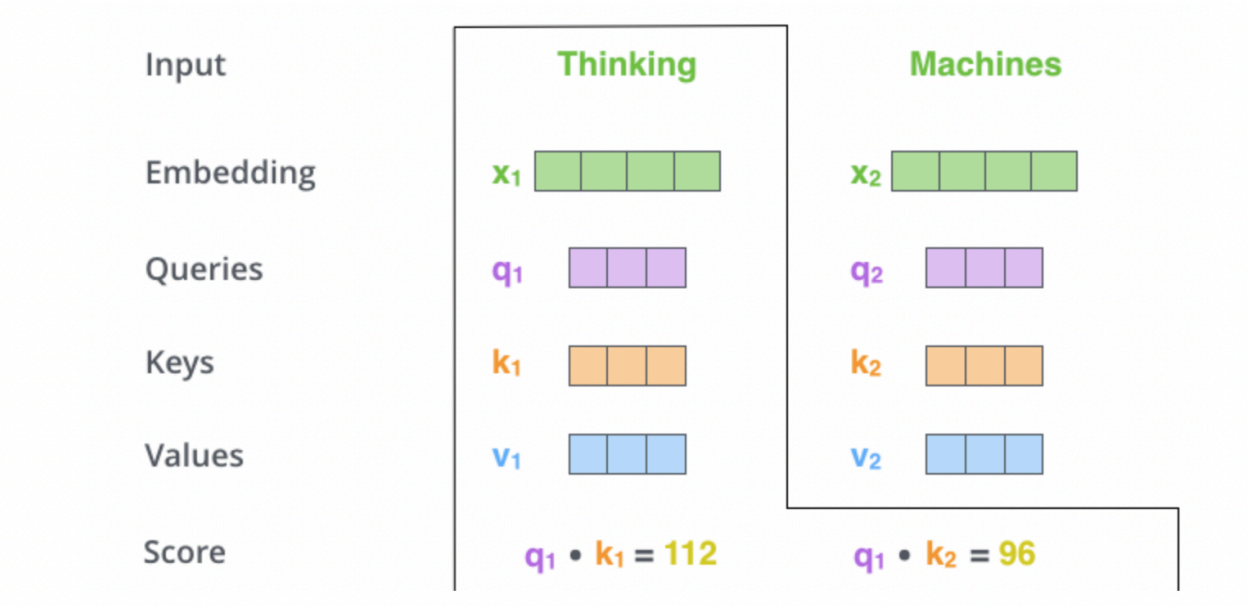
$Q = X @ W_Q$ # 查询向量（维度：64）

$K = X @ W_K$ # 键向量（维度：64）

$V = X @ W_V$ # 值向量（维度：64）

其中 W_Q 、 W_K 、 W_V 是模型训练学习到的权重矩阵。

Step 2: 计算注意力分数



假设我们在处理"Thinking"这个词：

- 计算"Thinking"的Q和所有词的K的点积（dot product）
- 这给出了"Thinking"对每个词的"关注程度"

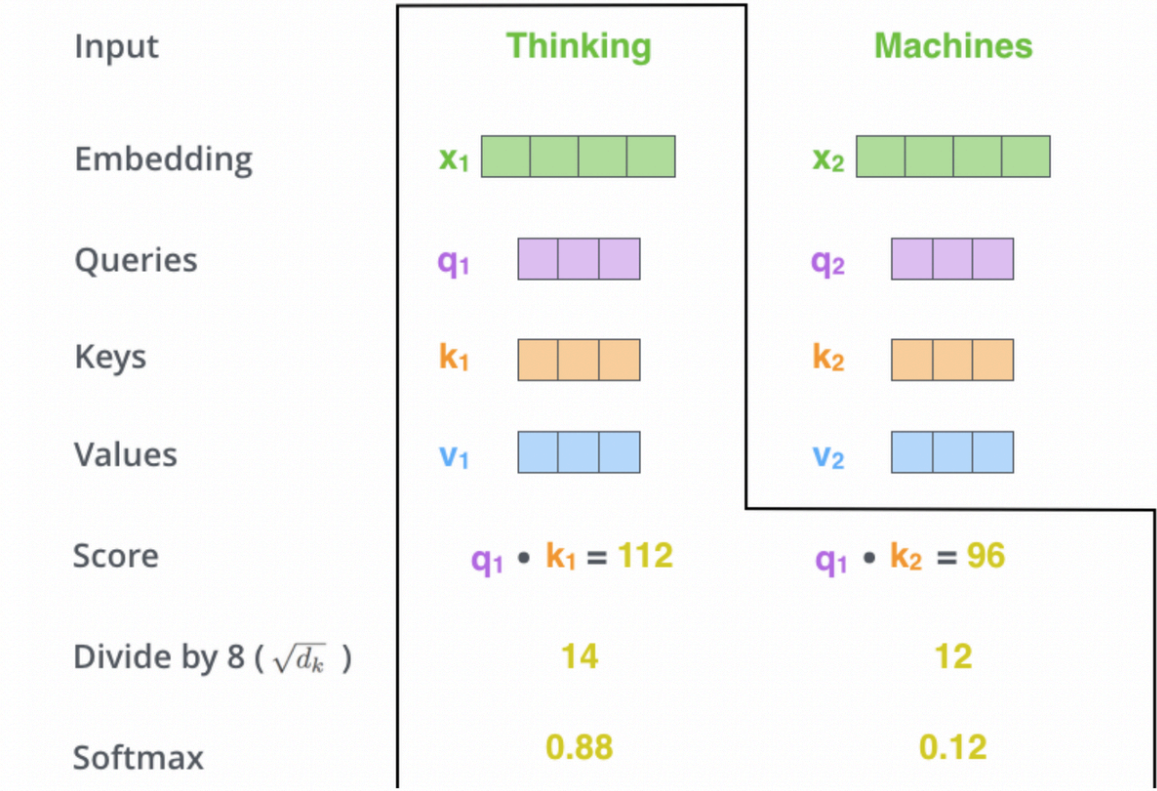
对于第一个词"Thinking"

```
score_1 = Q_thinking · K_thinking # 自己和自己
score_2 = Q_thinking · K_machines # 和"Machines"
```

为什么用点积？

- 点积大 → 两个向量方向相似 → 关系密切
- 点积小 → 两个向量方向不同 → 关系不大

Step 3 & 4：缩放 + Softmax归一化



缩放：除以 $\text{sqrt}(d_k)$ ，这里 $d_k=64$

`scaled_scores = scores / sqrt(64) = scores / 8`

Softmax：转换成概率分布（和为1）

`attention_weights = softmax(scaled_scores)`

为什么要除以8？

- 点积的结果可能很大，导致梯度消失
- 除以 $\text{sqrt}(d_k)$ 让数值更稳定

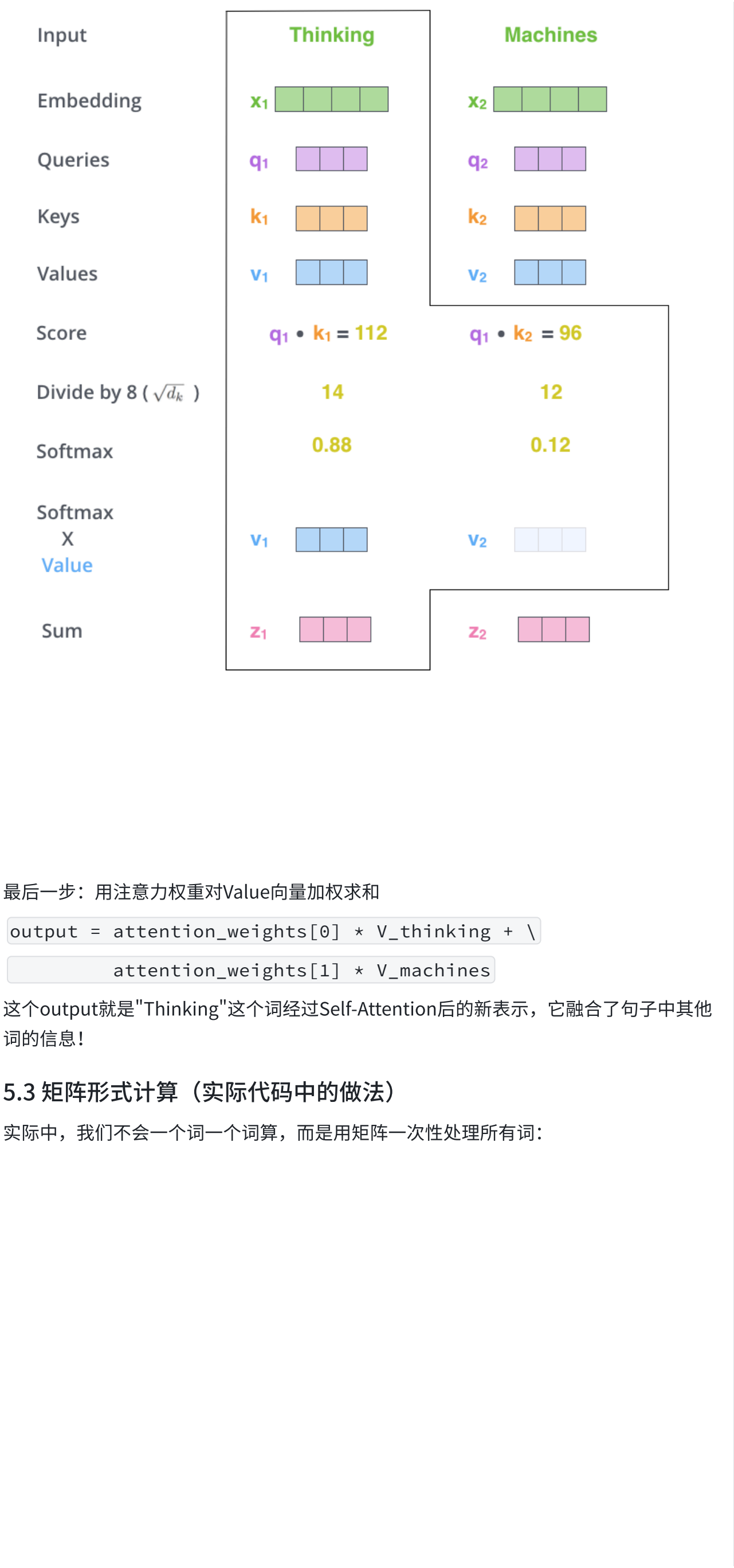
Softmax的作用：

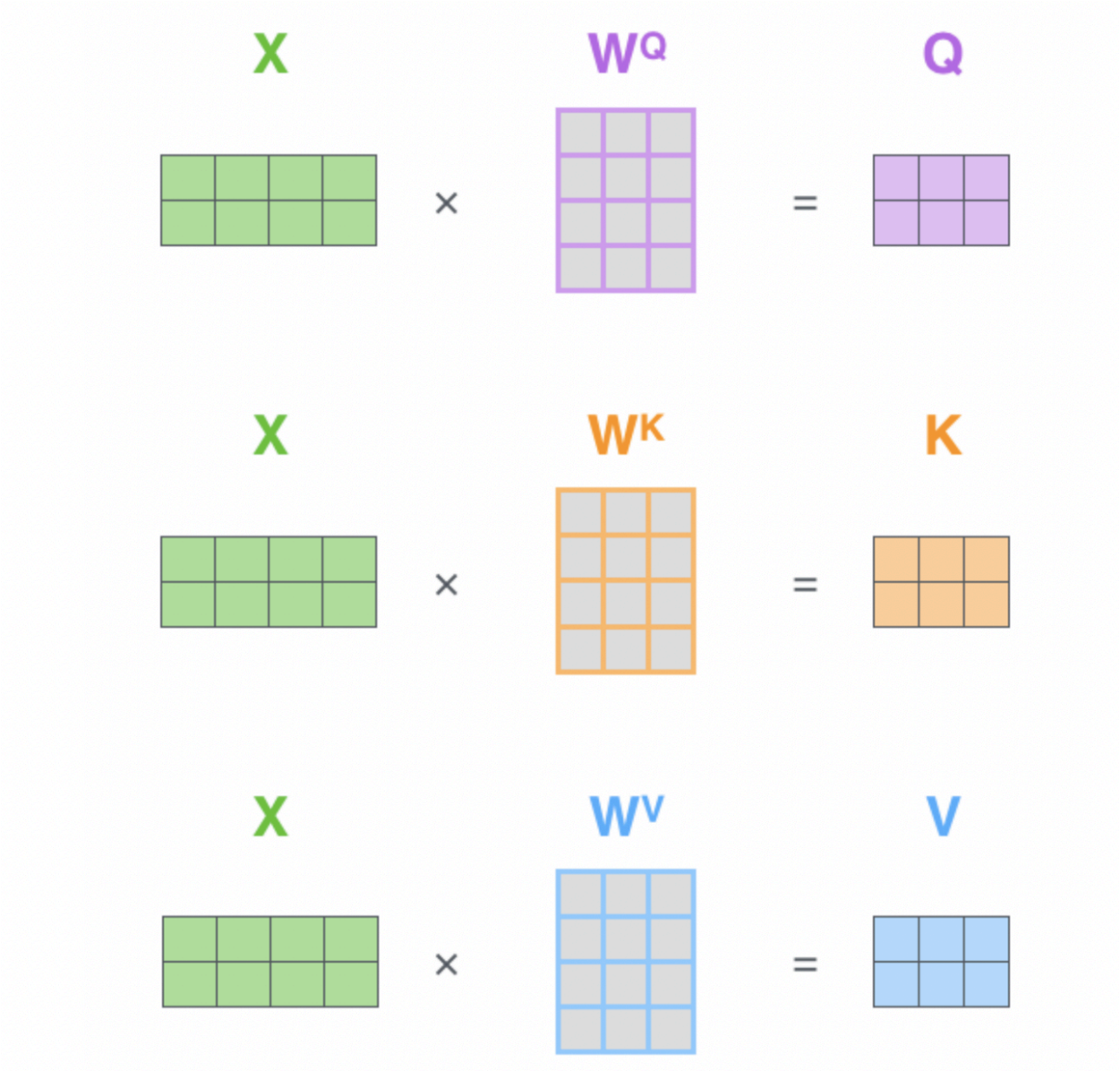
原始分数：[12, 8, 15]

Softmax后：[0.2, 0.1, 0.7] ← 变成概率，和为1

这样我们就知道了：对于"Thinking"这个词，应该给"Machines"0.7的权重。

Step 5 & 6：加权求和





Step 1: 计算Q、K、V矩阵

```
X: (seq_len, d_model) = (句子长度, 512)
W_Q, W_K, W_V: (d_model, d_k) = (512, 64)
Q = X @ W_Q # (seq_len, 64)
K = X @ W_K # (seq_len, 64)
V = X @ W_V # (seq_len, 64)
```

Step 2: 一个公式搞定所有！

$$\text{softmax}\left(\frac{\begin{matrix} \text{Q} \\ \begin{matrix} \square & \square & \square \\ \square & \square & \square \end{matrix} \end{matrix} \times \begin{matrix} \text{K}^T \\ \begin{matrix} \square & \square \\ \square & \square \\ \square & \square \end{matrix} \end{matrix}}{\sqrt{d_k}}\right) \begin{matrix} \text{V} \\ \begin{matrix} \square & \square & \square \\ \square & \square & \square \end{matrix} \end{matrix}$$

$$= \begin{matrix} \text{Z} \\ \begin{matrix} \square & \square & \square \\ \square & \square & \square \end{matrix} \end{matrix}$$

$$\text{Attention}(\text{Q}, \text{K}, \text{V}) = \text{softmax}(\text{Q} @ \text{K}^T / \text{sqrt}(\text{d}_k)) @ \text{V}$$

这一个公式包含了之前所有步骤：

1. $\text{Q} @ \text{K}^T$ → 计算所有词对之间的分数
2. $/ \text{sqrt}(\text{d}_k)$ → 缩放
3. $\text{softmax}(\dots)$ → 归一化成概率
4. $@ \text{V}$ → 加权求和

维度变化：

Q : (seq_len, d_k)

K^T : (d_k, seq_len)

$\text{Q} @ \text{K}^T$: (seq_len, seq_len) ← 注意力矩阵！

softmax : (seq_len, seq_len)

$@ \text{V}$: (seq_len, d_k) ← 输出维度

5.4 真实案例演示

让我们用一个真实例子走一遍流程：

输入句子： "I love AI" (3个词)

假设参数：

- 词向量维度 $\text{d_model} = 512$
- Q/K/V 维度 $\text{d}_k = 64$

数据流：

第1步：Embedding

"I" → x1: [0.2, 0.5, ..., 0.3] # 512维

"love" → x2: [0.8, 0.1, ..., 0.7] # 512维

"AI" → x3: [0.4, 0.9, ..., 0.2] # 512维

第2步：生成Q、K、V

$\text{q1} = \text{x1} @ \text{W}_Q \rightarrow [0.1, 0.3, \dots, 0.5]$ # 64维

$\text{k1} = \text{x1} @ \text{W}_K \rightarrow [0.2, 0.1, \dots, 0.8]$ # 64维

$\text{v1} = \text{x1} @ \text{W}_V \rightarrow [0.5, 0.4, \dots, 0.2]$ # 64维

同样处理 love、AI

第3步：计算attention scores (以"I"为例)

score_I_I = q1 · k1 = 32.5

score_I_love = q1 · k2 = 28.3

score_I_AI = q1 · k3 = 15.7

第4步：缩放

scaled_scores = [32.5/8, 28.3/8, 15.7/8] = [4.06, 3.54, 1.96]

第5步：Softmax

weights = softmax([4.06, 3.54, 1.96]) = [0.51, 0.41, 0.08]

意思是："I"主要关注自己(51%)和love(41%)，很少关注AI(8%)

第6步：加权求和

output_I = 0.51*v1 + 0.41*v2 + 0.08*v3

最终，output_I 就是"I"这个词在这个句子中的新表示，它已经融入了"love"和"AI"的上下文信息！

六、多头注意力（Multi-Head Attention）

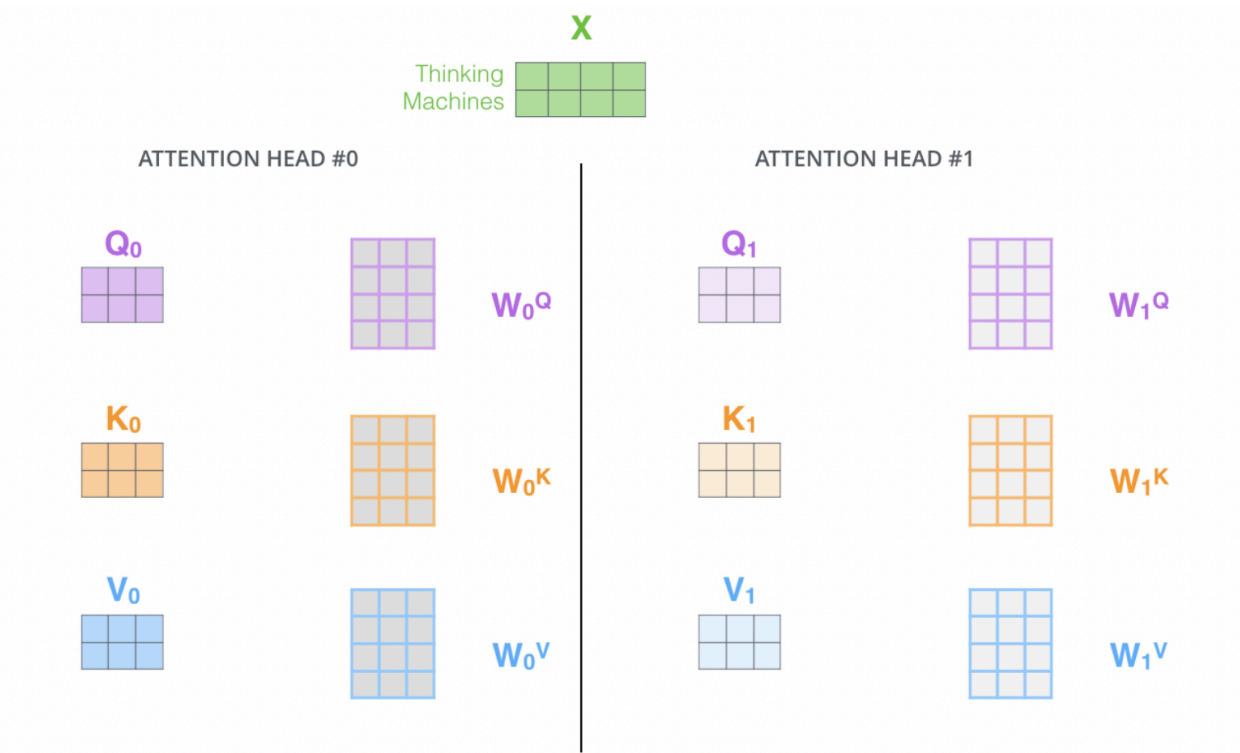
6.1 为什么需要多个头？

单头注意力有个问题：它只能学习一种关系模式。

但语言中的关系是多样的：

- 语法关系："主语-动词"、"动词-宾语"
- 语义关系："同义词"、"反义词"
- 位置关系："相邻词"、"远距离依赖"

多头注意力的思想： 让模型同时学习多种不同的关系！



6.2 多头注意力的实现

原理很简单：并行运行h个不同的注意力机制

Transformer使用8个注意力头

`h = 8`

每个头有自己的`W_Q`、`W_K`、`W_V`

```
for i in range(h):
```

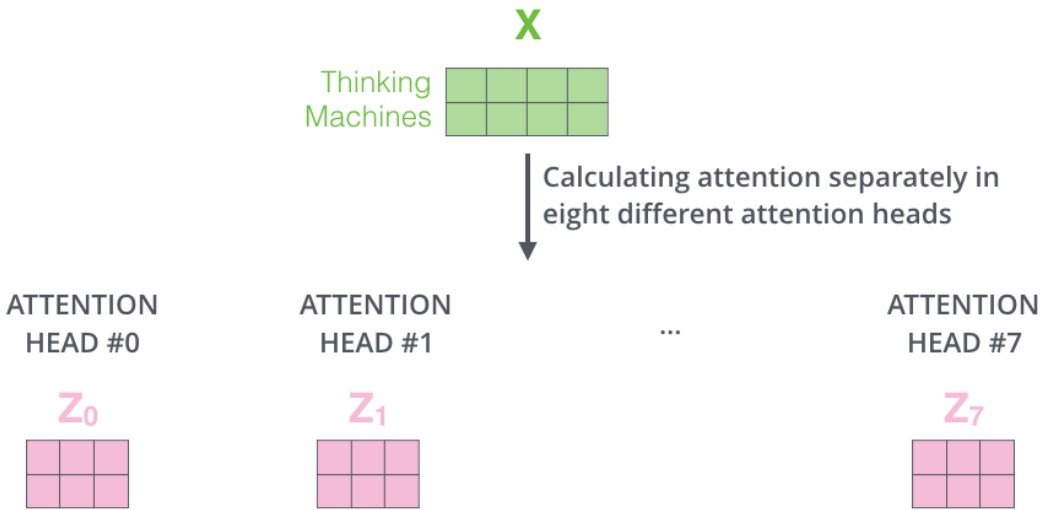
```
    Q_i = X @ W_Q_i
```

```
    K_i = X @ W_K_i
```

```
    V_i = X @ W_V_i
```

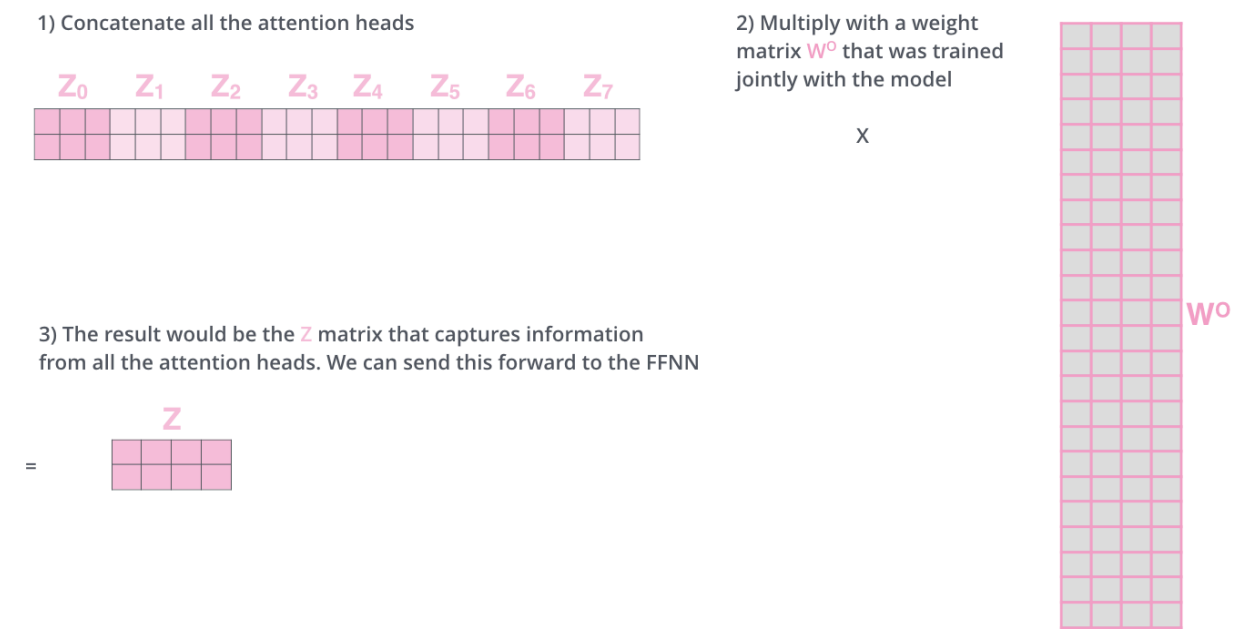
```
    # 独立计算attention
```

```
    Z_i = Attention(Q_i, K_i, V_i)
```



最后得到8个输出：Z₀, Z₁, ..., Z₇

6.3 合并多头输出



问题：前馈网络期望一个矩阵输入，但我们现在有8个！

解决办法：拼接 + 线性变换

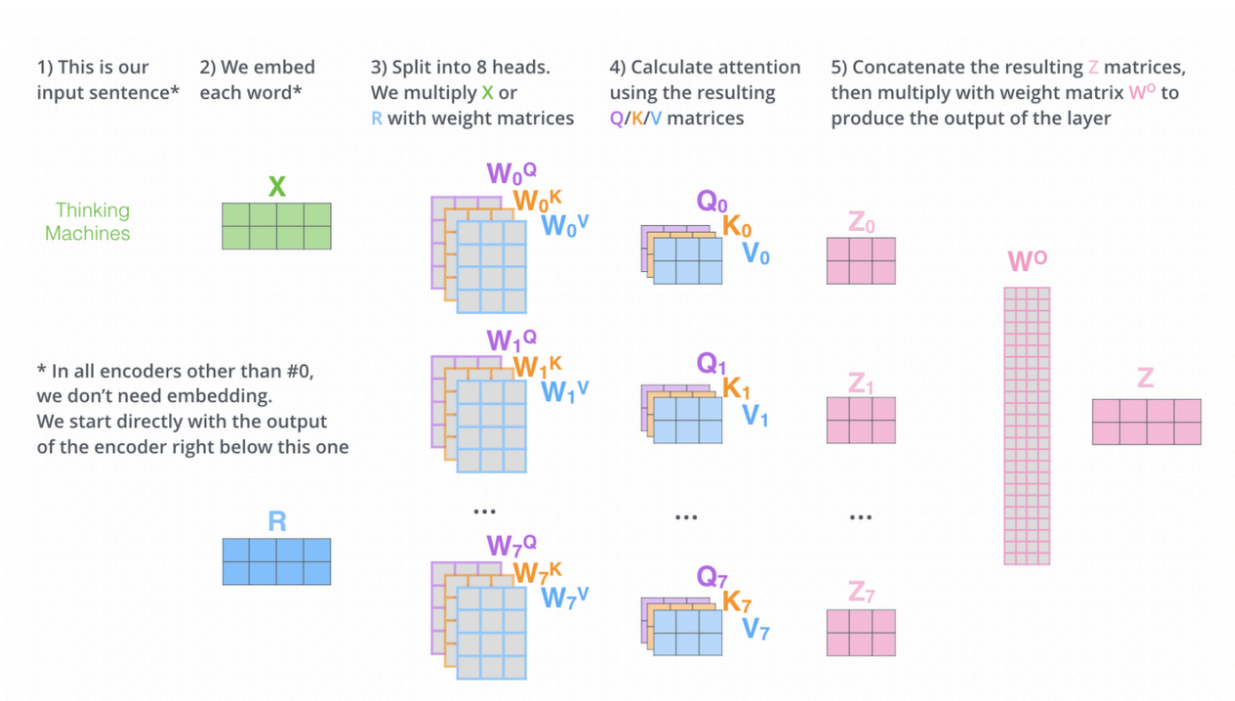
拼接所有头的输出

```
Z_concat = concat(Z_0, Z_1, ..., Z_7) # (seq_len, 8*d_k)
```

线性变换回原始维度

```
Z_final = Z_concat @ W_0 # (seq_len, d_model)
```

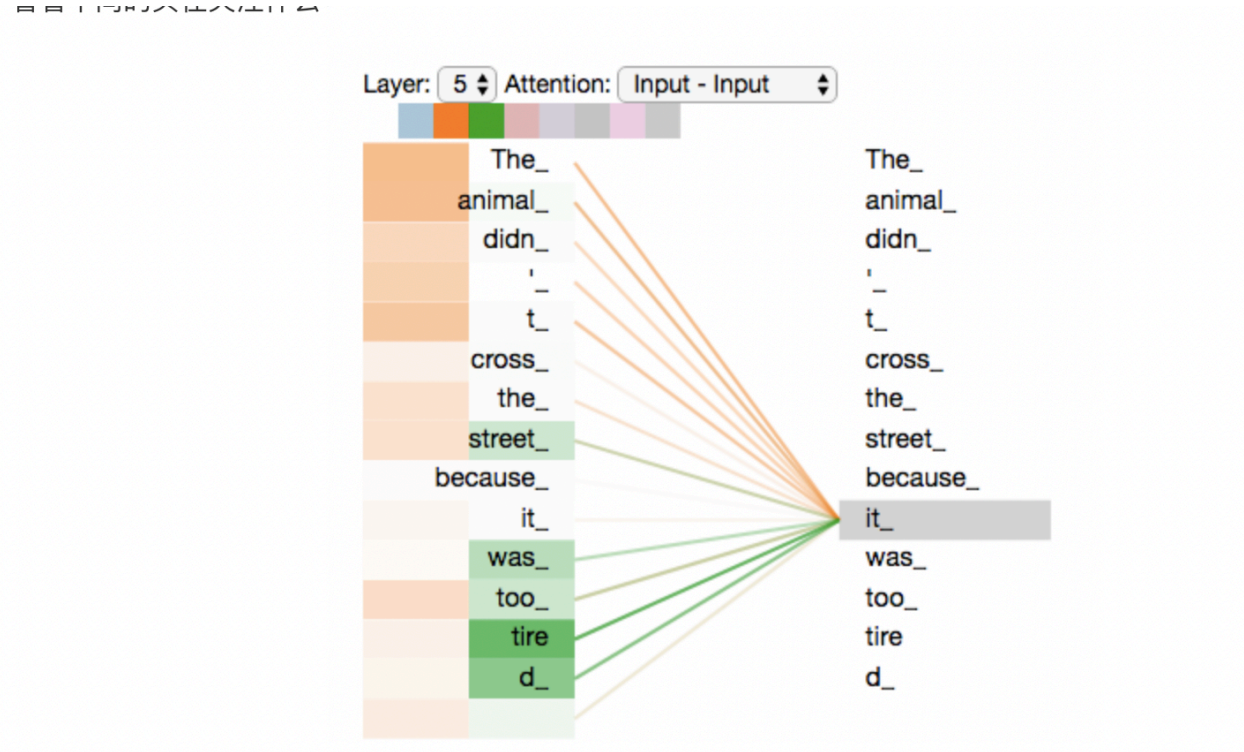
6.4 完整流程图



- 1. 输入X分别传给8个注意力头
- 2. 每个头独立计算attention
- 3. 拼接8个头的输出
- 4. 线性变换得到最终结果

6.5 多头注意力的可视化

看看不同的头在关注什么：



当模型处理"it"这个词时：

- 头1： 主要关注"animal"（语义关系）
- 头2： 关注"tired"（属性关系）
- 其他头关注不同的语法和语义模式

这就是为什么多头注意力这么强大——它能同时从多个角度理解句子！

七、位置编码（Positional Encoding）

7.1 为什么需要位置信息？

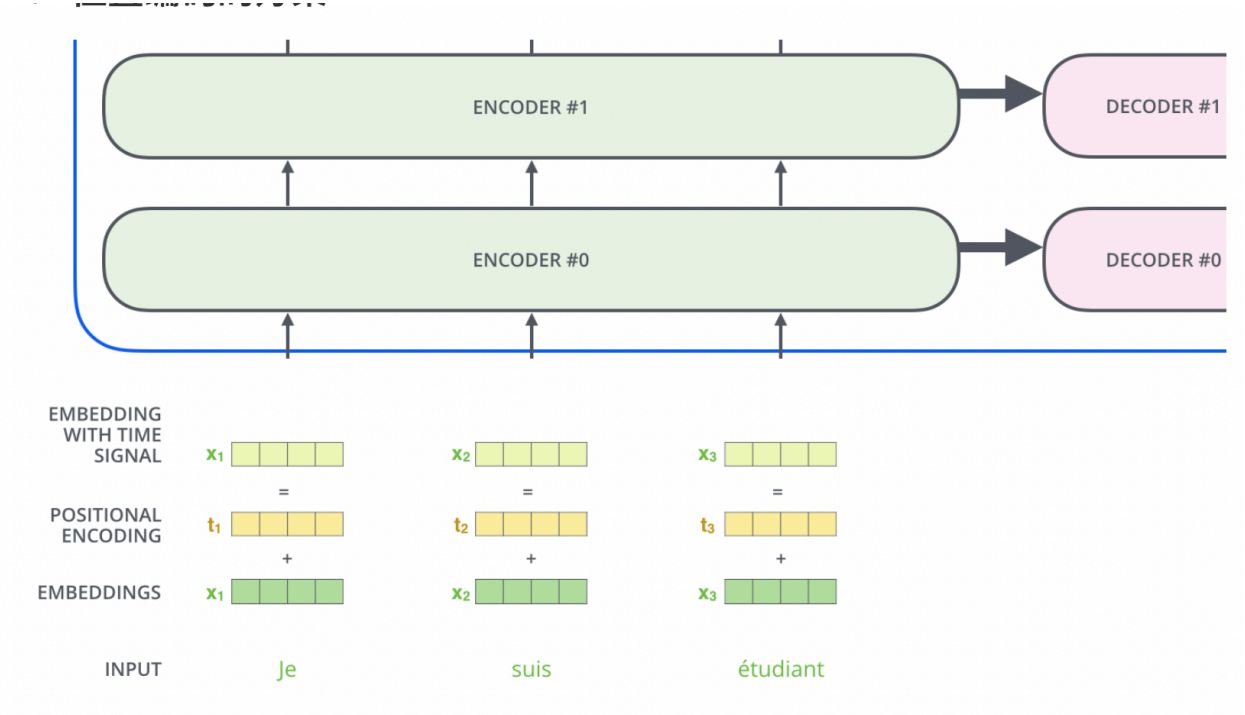
Self-Attention有个严重问题：它不知道词的顺序！

想象一下：

- "狗咬了人" 和 "人咬了狗" 在Self-Attention眼里是一样的！
- 因为注意力只看词之间的关系，不管它们的位置

但词的顺序在语言中太重要了！

7.2 位置编码的方案



解决办法：在词向量中加入位置信息

```
# 最终输入 = 词向量 + 位置向量
input_embedding = word_embedding + positional_encoding
```

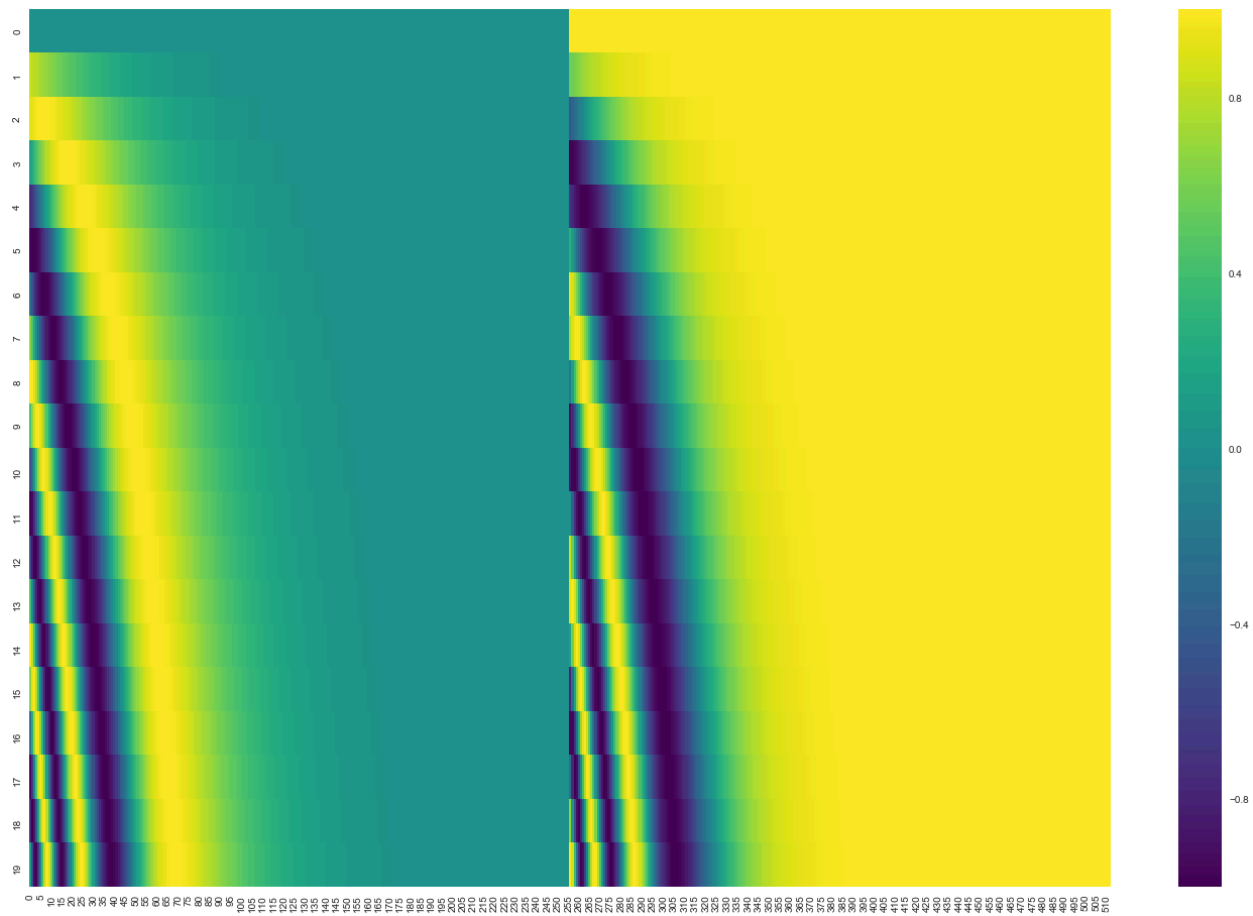
位置编码的公式：

```
PE(pos, 2i) = sin(pos / 10000^(2i/d_model))
PE(pos, 2i+1) = cos(pos / 10000^(2i/d_model))
```

其中：

- pos: 词的位置 (0, 1, 2, ...)
- i: 向量维度的索引
- d_model: 词向量维度 (512)

7.3 位置编码可视化



这是20个位置、512维的位置编码：

- 每一行是一个位置的编码
- 颜色代表数值大小
- 可以看出明显的周期性模式

为什么用sin/cos函数？

1. 值在[-1, 1]之间，不会太大
2. 对于任意长度的句子都能生成编码
3. 相对位置关系可以通过简单的线性变换得到

八、QKV架构细节深入剖析

8.1 为什么是Query、Key、Value？

这个命名来自于信息检索领域，让我们用一个完整类比理解：

场景：你在YouTube搜视频

1. 你输入搜索词："transformer教程" | ← Query（查询）

↓

2. YouTube数据库里每个视频都有标签： |

视频1: "AI, 机器学习, 神经网络"	← Key (索引)
视频2: "transformer, NLP, 注意力"	← Key
视频3: "做饭, 美食, 教程"	← Key

↓

3. 匹配度计算：	
视频1: 匹配度 0.2 (有点相关)	
视频2: 匹配度 0.9 (非常相关!)	
视频3: 匹配度 0.0 (不相关)	

↓

4. 返回视频内容 (根据匹配度加权)：	
主要展示视频2的内容(90%)	← Value (内容)
少量展示视频1的内容(10%)	← Value
不展示视频3	

对应到Self-Attention：

- **Query:** 当前词想要查找什么信息
- **Key:** 每个词能被什么样的查询匹配到
- **Value:** 每个词实际包含的信息内容

8.2 QKV的维度设计

```
# 典型参数
d_model = 512      # 词向量维度
d_k = d_v = 64     # QKV维度
h = 8              # 注意力头数

# 关键点: d_model = h * d_k
# 512 = 8 * 64
```

为什么QKV维度（64）比词向量（512）小？

1. 计算效率： 注意力计算复杂度是 $O(n^2)$ ， 维度越小越快
2. 多头注意力： 8个头 × 64维 = 512维， 刚好等于原始维度
3. 信息压缩： 强迫模型学习最重要的特征

8.3 QKV变换的详细过程

单个词的变换：

假设词"love"的embedding

```
x_love = [0.8, 0.1, 0.3, ..., 0.7] # 512维
```

三个权重矩阵 (512×64)

```
W_Q = [[0.1, 0.2, ..., 0.5],  
        [0.3, 0.1, ..., 0.8],  
        ...] # 512行64列
```

```
W_K = [...] # 512×64
```

```
W_V = [...] # 512×64
```

矩阵乘法

```
q_love = x_love @ W_Q # (1, 512) @ (512, 64) = (1, 64)
```

```
k_love = x_love @ W_K # (1, 64)
```

```
v_love = x_love @ W_V # (1, 64)
```

整个句子的变换：

句子矩阵：每行是一个词

```
X = [[x_I, # 512维  
      [x_love], # 512维  
      [x_AI]] # 512维
```

X的形状：(3, 512)

批量计算

```
Q = X @ W_Q # (3, 512) @ (512, 64) = (3, 64)
```

```
K = X @ W_K # (3, 64)
```

```
V = X @ W_V # (3, 64)
```

结果：

```
Q[0] = "I"的query向量
```

```
Q[1] = "love"的query向量
```

```
Q[2] = "AI"的query向量
```

8.4 实际代码实现

```
import torch
```

```
import torch.nn as nn
```

```
class SelfAttention(nn.Module):
```

```
    def __init__(self, d_model=512, d_k=64):
```

```
        super().__init__()
```

```
        self.d_k = d_k
```

```
        # 定义三个线性变换
```

```
        self.W_Q = nn.Linear(d_model, d_k, bias=False)
```

```

        self.W_K = nn.Linear(d_model, d_k, bias=False)

        self.W_V = nn.Linear(d_model, d_k, bias=False)

    def forward(self, X):
        """
        X: (batch_size, seq_len, d_model)
        输出: (batch_size, seq_len, d_k)
        """
        # Step 1: 生成Q、K、V
        Q = self.W_Q(X) # (batch, seq_len, d_k)
        K = self.W_K(X) # (batch, seq_len, d_k)
        V = self.W_V(X) # (batch, seq_len, d_k)

        # Step 2: 计算attention scores
        scores = torch.matmul(Q, K.transpose(-2, -1)) #
(batch, seq_len, seq_len)

        scores = scores / (self.d_k ** 0.5) # 缩放

        # Step 3: Softmax
        attention_weights = torch.softmax(scores, dim=-1)

        # Step 4: 加权求和
        output = torch.matmul(attention_weights, V) #
(batch, seq_len, d_k)

        return output, attention_weights

```

九、Attention输出计算的完整演示

9.1 手算一个简单例子

假设我们有一个极简化的例子（方便手算）：

输入： 2个词，维度为4

词向量（实际是512维，这里简化成4维）

$X = \begin{bmatrix} 1 & 0 & 1 & 0 \end{bmatrix}$, # "I"

$\begin{bmatrix} 0 & 2 & 0 & 2 \end{bmatrix}$ # "love"

权重矩阵（实际是512×64，这里简化成4×2）

$W_Q = \begin{bmatrix} 1 & 0 \end{bmatrix}$,

$\begin{bmatrix} 0 & 1 \end{bmatrix}$,

$\begin{bmatrix} 1 & 0 \end{bmatrix}$,

$\begin{bmatrix} 0 & 1 \end{bmatrix}$

$W_K = \begin{bmatrix} 0 & 1 \end{bmatrix}$,

$\begin{bmatrix} 1 & 0 \end{bmatrix}$,

$\begin{bmatrix} 0 & 1 \end{bmatrix}$,

$\begin{bmatrix} 1 & 0 \end{bmatrix}$

$$W_V = \begin{bmatrix} 1 & 1 \\ 1 & 1 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}$$

Step 1: 计算Q、K、V

$$\begin{aligned} Q &= X @ W_Q \\ &= \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 2 & 0 & 2 \end{bmatrix} @ \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} 1*1+0*0+1*1+0*0 & 1*0+0*1+1*0+0*1 \\ 0*1+2*0+0*1+2*0 & 0*0+2*1+0*0+2*1 \end{bmatrix} \\ &= \begin{bmatrix} 2 & 0 \\ 0 & 4 \end{bmatrix} \end{aligned}$$

$$K = X @ W_K = \begin{bmatrix} 0 & 1 \\ 2 & 2 \end{bmatrix}$$

$$V = X @ W_V = \begin{bmatrix} 2 & 2 \\ 2 & 2 \end{bmatrix}$$

Step 2: 计算Attention Scores

$$\begin{aligned} Scores &= Q @ K^T \\ &= \begin{bmatrix} 2 & 0 \\ 0 & 4 \end{bmatrix} @ \begin{bmatrix} 0 & 2 \\ 1 & 2 \end{bmatrix} \\ &= \begin{bmatrix} 2*0+0*1 & 2*2+0*2 \\ 0*0+4*1 & 0*2+4*2 \end{bmatrix} \\ &= \begin{bmatrix} 0 & 4 \\ 4 & 8 \end{bmatrix} \end{aligned}$$

意义解读：

- Scores0 = 0: "I"对"I"的关注度
- Scores0 = 4: "I"对"love"的关注度
- Scores1 = 4: "love"对"I"的关注度
- Scores1 = 8: "love"对"love"的关注度

Step 3: 缩放 (d_k=2)

$$\begin{aligned} Scaled &= Scores / \sqrt{2} = Scores / 1.414 \\ &= \begin{bmatrix} 0 & 2.83 \\ 2.83 & 5.66 \end{bmatrix} \end{aligned}$$

Step 4: Softmax

对每一行做softmax

$$\text{Row 0: } \begin{bmatrix} 0 & 2.83 \end{bmatrix}$$

1. 多头注意力 + 残差连接 + LayerNorm...



电商用户1221 7月17日 11:35

这里用的 Post-LN 后置归一化，当前主流是 Pre-LN 前置归一化，将最后一步的归一化提前到第一步。这样深层训练稳定，易收敛

exp: [1, 16.95]

sum: 17.95

softmax: [0.06, 0.94]

Row 1: [2.83, 5.66]

exp: [16.95, 287.28]

sum: 304.23

softmax: [0.06, 0.94]

Attention_Weights = [[0.06, 0.94],

[0.06, 0.94]]

意义:

- "I"这个词: 6%关注自己, 94%关注"love"
- "love"这个词: 6%关注"I", 94%关注自己

Step 5: 加权求和

Output = Attention_Weights @ V

= [[0.06, 0.94], @ [[2, 2],

[0.06, 0.94]] [2, 2]]

= [[0.06*2+0.94*2, 0.06*2+0.94*2],

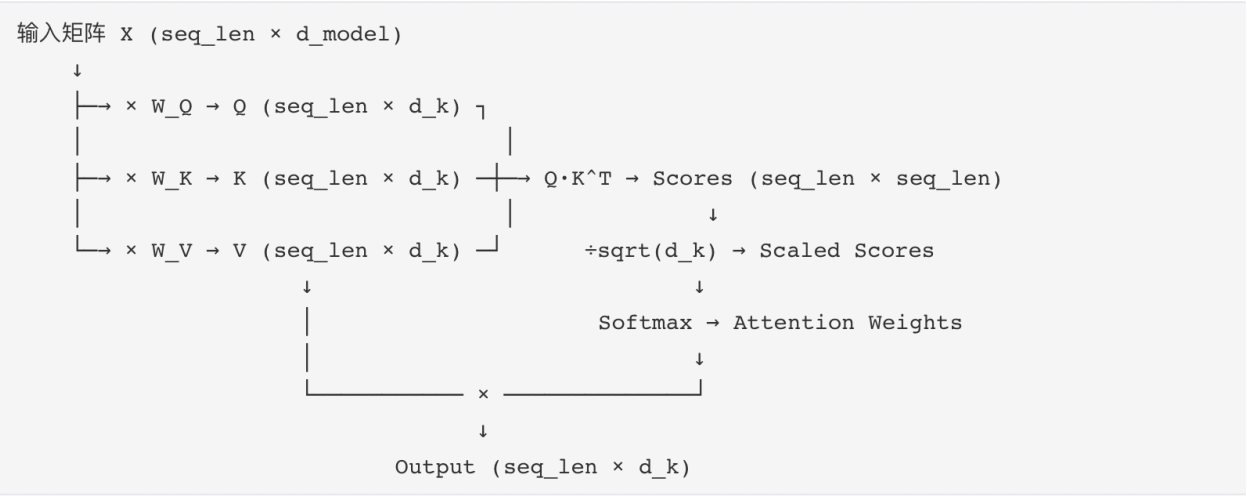
[0.06*2+0.94*2, 0.06*2+0.94*2]]

= [[2, 2],

[2, 2]]

9.2 完整架构图演示

让我们画一个完整的Attention计算流程图:



9.3 实际尺寸的计算

让我们用真实的Transformer参数走一遍:

```
# 参数设置
batch_size = 32      # 批大小
seq_len = 50         # 句子长度
d_model = 512        # 词向量维度
d_k = 64             # Q/K/V维度
num_heads = 8        # 注意力头数

# 输入
X: (32, 50, 512)

# 单头注意力
Q = X @ W_Q: (32, 50, 512) @ (512, 64) = (32, 50, 64)
K = X @ W_K: (32, 50, 64)
V = X @ W_V: (32, 50, 64)

# Attention计算
Scores = Q @ K^T: (32, 50, 64) @ (32, 64, 50) = (32, 50, 50)
Scaled = Scores / 8: (32, 50, 50)
Weights = softmax(Scaled): (32, 50, 50) ← 注意力矩阵!
Output = Weights @ V: (32, 50, 50) @ (32, 50, 64) = (32, 50, 64)

# 8个头的结果拼接
All_Heads_Output: (32, 50, 8*64) = (32, 50, 512)

# 最终线性变换
Final = All_Heads_Output @ W_O: (32, 50, 512)
```

9.4 可视化Attention矩阵

Attention Weights矩阵（seq_len × seq_len）是最关键的，它告诉我们每个词关注哪些词。

例子： "The cat sat on the mat"

```
The cat sat on the mat
The [0.8 0.1 0.0 0.0 0.1 0.0]
cat [0.2 0.5 0.2 0.0 0.0 0.1]
sat [0.0 0.3 0.4 0.2 0.0 0.1]
on  [0.0 0.0 0.3 0.4 0.2 0.1]
the [0.0 0.0 0.0 0.3 0.5 0.2]
mat [0.0 0.1 0.0 0.1 0.3 0.5]
```

解读：

- "cat"这个词：50%关注自己，20%关注"The"，20%关注"sat"
- "sat"这个词：40%关注自己，30%关注"cat"，20%关注"on"
- 可以看出动词"sat"关注了主语"cat"和介词"on"

十、实战：用PyTorch实现完整的Transformer Block

10.1 完整代码

```
import torch

import torch.nn as nn

import math


class MultiHeadAttention(nn.Module):

    """多头注意力层"""

    def __init__(self, d_model=512, num_heads=8):

        super().__init__()

        assert d_model % num_heads == 0, "d_model必须能被num_heads整除"

        self.d_model = d_model

        self.num_heads = num_heads

        self.d_k = d_model // num_heads # 每个头的维度

        # Q、K、V的权重矩阵（包含所有头）

        self.W_Q = nn.Linear(d_model, d_model)

        self.W_K = nn.Linear(d_model, d_model)

        self.W_V = nn.Linear(d_model, d_model)

        # 最后的线性变换

        self.W_O = nn.Linear(d_model, d_model)

    def split_heads(self, x, batch_size):

        """

        将最后一维分成(num_heads, d_k)

        x: (batch, seq_len, d_model)

        输出: (batch, num_heads, seq_len, d_k)

        """

        x = x.view(batch_size, -1, self.num_heads, self.d_k)

        return x.transpose(1, 2)

    def forward(self, Q, K, V, mask=None):

        """

        Q, K, V: (batch, seq_len, d_model)

        mask: (batch, seq_len, seq_len) 可选

        """

        batch_size = Q.shape[0]

        # 1. 线性变换

        Q = self.W_Q(Q) # (batch, seq_len, d_model)

        K = self.W_K(K)

        V = self.W_V(V)

        # 2. 分成多头

        Q = self.split_heads(Q, batch_size) # (batch,
```

```

        K = self.split_heads(K, batch_size)
        V = self.split_heads(V, batch_size)

        # 3. 计算attention
        scores = torch.matmul(Q, K.transpose(-2, -1)) #
(batch, num_heads, seq_len, seq_len)

        scores = scores / math.sqrt(self.d_k) # 缩放

        # 4. 应用mask (如果有)
        if mask is not None:
            scores = scores.masked_fill(mask == 0, -1e9)

        # 5. Softmax
        attention_weights = torch.softmax(scores, dim=-1)

        # 6. 加权求和
        output = torch.matmul(attention_weights, V) #
(batch, num_heads, seq_len, d_k)

        # 7. 合并多头
        output = output.transpose(1, 2).contiguous() #
(batch, seq_len, num_heads, d_k)

        output = output.view(batch_size, -1, self.d_model)
# (batch, seq_len, d_model)

        # 8. 最后的线性变换
        output = self.W_0(output)

        return output, attention_weights

```

```

class FeedForward(nn.Module):
    """前馈神经网络"""

    def __init__(self, d_model=512, d_ff=2048):
        super().__init__()

        self.linear1 = nn.Linear(d_model, d_ff)
        self.linear2 = nn.Linear(d_ff, d_model)
        self.relu = nn.ReLU()

    def forward(self, x):
        # x: (batch, seq_len, d_model)

        return self.linear2(self.relu(self.linear1(x)))

```

```

class TransformerBlock(nn.Module):
    """Transformer编码器块"""

    def __init__(self, d_model=512, num_heads=8, d_ff=2048,
dropout=0.1):
        super().__init__()

        # 多头注意力

```

```

        self.attention = MultiHeadAttention(d_model,
num_heads)

        # 前馈网络

        self.feed_forward = FeedForward(d_model, d_ff)

        # Layer Normalization

        self.norm1 = nn.LayerNorm(d_model)

        self.norm2 = nn.LayerNorm(d_model)

        # Dropout

        self.dropout = nn.Dropout(dropout)

        def forward(self, x, mask=None):

            """

            x: (batch, seq_len, d_model)

            mask: (batch, seq_len, seq_len)

            """

            # 1. 多头注意力 + 残差连接 + LayerNorm

            attn_output, attention_weights = self.attention(x,
x, x, mask)

            x = self.norm1(x + self.dropout(attn_output))

            # 2. 前馈网络 + 残差连接 + LayerNorm

            ff_output = self.feed_forward(x)

            x = self.norm2(x + self.dropout(ff_output))

            return x, attention_weights

```

使用示例

```

if name == "__main__":

    # 参数

    batch_size = 2

    seq_len = 10

    d_model = 512

    # 创建模型

    transformer_block = TransformerBlock(d_model=512,
num_heads=8)

    # 随机输入

    x = torch.randn(batch_size, seq_len, d_model)

    # 前向传播

    output, attention_weights = transformer_block(x)

    print(f"输入形状: {x.shape}")

    print(f"输出形状: {output.shape}")

    print(f"注意力权重形状: {attention_weights.shape}")

    # 输出:

    # 输入形状: torch.Size([2, 10, 512])

    # 输出形状: torch.Size([2, 10, 512])

```

```
# 注意力权重形状: torch.Size([2, 8, 10, 10])
```

10.2 代码详解

关键点1: split_heads函数

这个函数把(batch, seq_len, d_model)转换成(batch, num_heads, seq_len, d_k)

```
# 例子
x = torch.randn(2, 10, 512) # batch=2, seq_len=10, d_model=512
# 目标: 分成8个头, 每个头64维

x = x.view(2, 10, 8, 64) # (batch, seq_len, num_heads, d_k)
x = x.transpose(1, 2)     # (batch, num_heads, seq_len, d_k)
# 现在可以对每个头独立计算attention了!
```

关键点2: 残差连接 (Residual Connection)

```
# 不使用残差连接
x = self.attention(x)

# 使用残差连接 (重要!)
x = x + self.attention(x)
```

为什么重要?

- 防止梯度消失
- 允许更深的网络 (Transformer可以堆叠100+层)
- 加速训练

关键点3: Layer Normalization

归一化每个样本的每个位置

```
x = self.norm(x) # 在d_model维度上归一化
```

作用:

- 稳定训练
- 加速收敛